

EXPLAINABLE AI

Understanding, Evaluating, and Engineering
Explanations for Intelligent Systems



FROM BLACK BOXES TO ACCOUNTABLE UNDERSTANDING

DRAFT EDITION 2026

Copyright © [2026] Axiologic Research

All rights reserved.

KDP publishing rights held by Outfinity SRL (Iași, Romania).

Available for free at www.axiologic.net.

Author's Note

This work was created under the umbrella of Axiologic Research as part of two interconnected research programs, one dedicated to Artificial Intelligence and the other to Outfinitism, both led by Sinică Alboaie. To explore the details of how these two programs intersect and build upon each other, we invite readers to visit the Outfinity Venture Studio page at www.outfinity.ch.

While Artificial Intelligence language models were used to assist with text generation, the foundational philosophy, thematic structure, and analytical approach derive exclusively from the author's decades of dedicated study of social phenomena, social technologies, and genuine hard technologies resulting from various European and self funded research projects. Recently, within the Achilles research project, we have been deeply studying AI generated literature and its automated verification using neuro symbolic approaches; all the free books made available to the public are part of the suite of works we use to test our latest technologies.

Editorial scope of this edition. The research literature reflected in the manuscript was reviewed through 30 August 2026. The book is intentionally written without unnecessary mathematical formalism. When a mathematical object matters, the text explains what it means operationally, which assumption it carries, and how the associated claim could be tested or falsified.

Contents

Chapter titles only. Page numbers are intentionally omitted.

Chapter 1. The Explanatory Problem

Chapter 2. Interrogating Predictions

Chapter 3. Testing Explanations in the Real World

Chapter 4. Explaining Foundation Models

Chapter 5. From Models to Systems

Chapter 6. Toward Executable Understanding

Chapter 1. The Explanatory Problem

Explainable AI is often introduced as a toolbox: a saliency method here, a Shapley method there, perhaps a counterfactual generator and a dashboard. That framing is convenient for software libraries and disastrous for reasoning. The difficult part is not producing an explanatory artifact. The difficult part is deciding what claim the artifact is allowed to support. This chapter therefore starts before the algorithms. It follows the moment at which a prediction becomes evidence for a decision, shows why different stakeholders are asking genuinely different questions, and develops an explanation contract that makes those questions testable. Only after the semantics are explicit does the chapter ask whether a black box is needed at all. The result is a foundation for the rest of the book: explanation is treated as an engineered evidence interface, not as decoration around a predictor.

QUESTION TO CARRY THROUGH THE CHAPTER — What explanatory claim are we making, and what evidence would actually support it?

1.1 When accuracy stops being enough

Imagine two teams deploying the same risk model. The first celebrates a small increase in AUROC and publishes a dashboard with a SHAP plot. The second discovers, during a complaint investigation, that nobody can answer a basic question: did the model reject this person because of information that should have been irrelevant, because of a legitimate but surprising interaction, or because the input was outside the region where the model had been validated? Both teams possess the same predictive model. Only one has treated explanation as an engineering requirement. This distinction is the practical beginning of XAI.

Accuracy stops being enough whenever a prediction becomes a decision, evidence for a decision, or a component inside a longer process. In debugging, an engineer needs an explanation that is sensitive to the trained model rather than merely visually plausible. In medicine, a clinician may need to know whether a finding is consistent with domain knowledge, whether the system is uncertain, and whether a recommendation survives a plausible change in measurement conditions.

In an automated agent, an operator may need to reconstruct which retrieved document triggered a tool call and whether a different source would have changed the action. Each setting changes the explanatory target.

A useful mental model is to treat prediction and explanation as two different interfaces. The prediction interface answers a task question: class, score, text continuation, ranking, action. The explanation interface answers a question about the system: dependence, contrast, mechanism, provenance, uncertainty, or policy compliance. The second interface therefore cannot be specified by saying only “use SHAP” or “show the reasoning.” It needs requirements just as an API does. Haufe and colleagues’ 2026 call for formalization is important because it gives theoretical language to a problem engineers repeatedly encounter: a method may return numbers that are mathematically well defined while the user interprets those numbers as answers to a different question.

The story of modern XAI can be read as a sequence of attempts to close this gap. Early post-hoc methods made black boxes locally inspectable. Later work showed that explanations could remain plausible under model randomization, could be manipulated adversarially, or could collapse under small changes in baseline and background distribution. Counterfactual and recourse research exposed the difference between changing a model input and changing a person’s feasible options. Human-centered work showed that a technically faithful explanation can still create the wrong mental model. Mechanistic interpretability now goes deeper, asking whether internal computations can be identified and causally tested. The field is not abandoning older tools; it is learning to place each tool inside a more explicit evidential hierarchy.

Suppose a credit model rejects an application. A data scientist may ask which input variables most influenced the score. A regulator may ask whether the decision depended on a prohibited attribute or a proxy. The applicant may ask what could realistically be changed to obtain a different outcome. A model developer may ask whether the classifier learned a shortcut. A security engineer may ask whether the model can be manipulated into producing a benign-looking explanation while retaining undesirable behavior. These are not different phrasings of one question. They request different objects, require different evidence, and fail in different ways.

The purpose of this table is to prevent the word explanation from collapsing distinct engineering tasks into one interface. The first column names the operational question; the second states the kind of evidence the system must eventually support.

Table 1.1 — Different explanatory questions require different evidence.

Concept / object	Engineering meaning
Debugging	Evidence that identifies model behavior or failure conditions and reacts when the trained model changes.
Decision support	Evidence that helps a person choose an action while exposing uncertainty and known limits.
Recourse	Evidence that links a changed decision to feasible interventions in the world, not only editable inputs.
Audit	Evidence that behavior can be reproduced, compared across groups, and checked against policy or legal requirements.

The table also shows why a single visual explanation cannot satisfy every stakeholder. A debugging artifact may be technically excellent and still be useless for recourse or audit because those tasks require different assumptions and downstream evidence.

This is the first source of confusion in XAI. The word explanation is used for feature importance, local sensitivity, global approximation, counterfactual change, causal effect, training-data influence, concept dependence, example similarity, attention patterns, generated rationales, and hypotheses about internal mechanisms. A method can be correct for one of these meanings and misleading for another. Treating the outputs as interchangeable is therefore a semantic error before it becomes a statistical one.

The problem-first view is now receiving explicit theoretical support. Haufe and colleagues argue that much of XAI developed algorithm-first: researchers created attribution techniques and later attached broad explanatory interpretations to their outputs. Their 2026 perspective calls instead for formally defining the explanatory problem, specifying correctness criteria, and validating against controlled or synthetic ground truth. The engineering translation is simple: the API of an explainer should include its semantics, not only its function signature.

Terminology remains inconsistent across the literature, so this book adopts an operational convention. Interpretability refers to properties that make the predictive mechanism or representation understandable enough to inspect directly. Explainability is broader and includes post-hoc procedures that interrogate or approximate systems whose native

computation is not readily understandable. Transparency refers to the amount and kind of information exposed about the model, its data, its training process, and its operation. These terms overlap, but keeping them distinct prevents several common mistakes.

A sparse linear model can be intrinsically interpretable when its variables are meaningful and the number of interactions remains manageable. A gradient-boosted tree ensemble may be opaque as a whole but admit high-quality local or global analyses. A large language model can be highly transparent in the narrow sense that its weights are downloadable while remaining almost completely unintelligible mechanistically. Conversely, a proprietary model can expose carefully validated explanation interfaces without exposing weights. Open weights, interpretability, and explainability are therefore different dimensions.

Local explanations concern one prediction or a narrow neighborhood. Global explanations aim to characterize the model over a larger input domain. Regional explanations target a cohort, cluster, or operational regime. These scopes cannot be collapsed into one another without assumptions. A locally accurate linear surrogate may describe a decision boundary well around one instance while being useless as a global model. Conversely, a global feature-importance ranking can hide heterogeneous local behavior, interactions, and subgroup failures.

The scope matters operationally. When debugging an individual failure, local analysis is appropriate. When writing a model card, validating policy constraints, or evaluating systematic bias, global and subgroup analyses are essential. In production, a useful architecture often combines all three: global model-level diagnostics, regional monitoring for cohorts and operational modes, and local explanations for selected decisions.

The practical consequence is that XAI begins by naming the question, not by choosing a library. Once the explanatory target is explicit, the next task is to specify what evidence would actually count as an answer.

1.2 The explanation contract: target, audience, claim, and evidence

The phrase explanation contract is not a new algorithm. It is a way to stop an organization from changing the meaning of an explanation after seeing the output.

The cleanest way to prevent category errors is to write an explanation contract before choosing a method. The contract begins with four questions. What is the object being explained? Who will consume the explanation? What claim is the explanation allowed to support? What

evidence would make that claim credible? These questions sound administrative, but they determine the mathematics. A local sensitivity analysis, a causal recourse recommendation, and a mechanistic circuit hypothesis can all be produced for the same model and all be valid while answering different questions.

Consider an intensive-care deterioration model. For a data scientist, “why did the score increase?” may reasonably mean which observed variables changed the prediction most in the current neighborhood. For a physician, the useful question may be whether the model is reacting to clinically meaningful physiology or to artifacts of documentation. For a hospital governance board, the question may be whether performance and explanation quality differ systematically across patient groups. For a patient, the relevant explanation may concern what the system contributed to a decision and where human judgment entered. One explainer cannot satisfy all four audiences simply by changing the chart title.

The contract also limits interpretation. If SHAP values are computed using a particular background distribution, the documentation should state that the values allocate a prediction relative to that reference under a specified feature-missingness convention. It should not silently upgrade the result into a causal statement. If an activation patching experiment shows that replacing one internal representation restores a correct answer, the claim should be phrased as evidence of causal relevance under that intervention, not as complete reverse engineering of the reasoning process. Precision in language is not pedantry; it is how XAI prevents explanatory outputs from becoming confidence theater.

This leads to a simple evidence ladder. Descriptive evidence shows patterns in outputs or internal states. Perturbational evidence asks whether controlled changes alter the prediction. Causal internal evidence intervenes on representations or components. Formal evidence proves selected properties under explicit assumptions. Human evidence shows that an explanation improves a real task without producing harmful over-trust. No single rung dominates every application. The purpose of the contract is to choose the weakest evidence that is sufficient for the actual decision, then state clearly what remains unproven.

A practical way to write the contract is to imagine how the explanation could be misused. If the output is a ranking of influential features, could a reader treat the ranking as a causal prescription? If the output is a generated rationale, could a reviewer assume that every mentioned fact was internally decisive? If the output is a local surrogate, could a product team summarize it as a global rule? Writing these foreseeable

misinterpretations next to the intended claim forces the design team to expose semantic boundaries before deployment.

The diagram captures the logical order that should precede method selection. An explanatory question is translated into a target, the claim and its assumptions are stated, validation evidence is defined, and only then is the result allowed to influence a decision or action.

The explanation contract

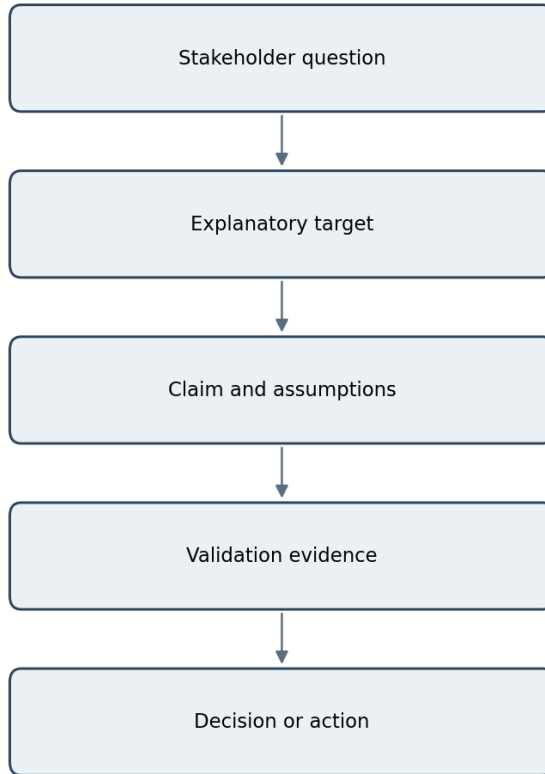


Figure 1.1 — The explanation contract turns a vague request for transparency into a sequence of explicit engineering decisions.

Notice what is intentionally absent from the first box: the name of an XAI algorithm. SHAP, LIME, Integrated Gradients, counterfactual search, probes, or mechanistic tools enter only after the explanatory target is known. This order is the simplest defense against using a technically correct output to answer the wrong question.

The contract should also specify abstention. Explanation systems are often designed as if they must always return an answer, yet there are situations in which the honest output is “the available evidence is unstable.” A local explanation near a sharp decision boundary may vary dramatically under tiny perturbations. A counterfactual may depend on an uncertain causal graph. A mechanistic feature label may fit the sampled examples but fail on a new domain. An explicit abstention state is therefore part of explanatory quality, just as abstention or uncertainty can be part of predictive quality.

Finally, the contract gives reviewers something concrete to test. They can ask whether the chosen method truly produces the declared object, whether the evidence is strong enough for the declared claim, whether the target audience can use it correctly, and whether the explanation is retained with enough provenance to be reproduced. This is the point where XAI stops being a collection of plots and becomes an engineering interface with testable requirements.

A useful way to understand the explanation contract is to compare it with a measurement protocol in experimental science. A thermometer is not trusted because it displays a number; it is trusted because the quantity, unit, operating range, calibration procedure, and failure conditions are specified. XAI outputs deserve the same discipline. A SHAP vector, counterfactual, or activation label is only the displayed number. The contract states what quantity that number is intended to represent, which interventions or comparisons define it, and which observations would show that the representation has become unreliable.

This framing also exposes a common category error. An organization may validate that an attribution method is stable and then use the result as evidence of fairness. Stability and fairness are different properties. It may validate a counterfactual generator against a classifier and then present the counterfactual as feasible advice to a person. Boundary crossing and feasible recourse are different properties. A contract prevents these semantic upgrades from happening silently because the claim is written before the artifact is generated.

The contract is especially valuable in multi-team environments. Model developers, product teams, compliance staff, and domain experts often use the same word—explanation—while expecting different evidence. Recording the audience and intended action forces those expectations into the open. It also makes later disagreements productive: the teams can argue about an explicit claim and acceptance test rather than about whether a visualization 'looks convincing.'

Figure 1.1. The explanation contract: the method comes after the question and the evidential claim.

One of the most consequential distinctions in applied XAI is between explaining a predictive model and explaining the real-world system represented by the data. A model may rely strongly on an input variable because that variable is predictive in the training distribution. This does not establish that changing the variable would change the real-world outcome. Feature attributions describe a function under a background distribution and a particular masking or conditioning convention. Causal explanations require assumptions about interventions, confounding, and the structure of the underlying process.

A hospital model may assign high predictive importance to a treatment because the treatment is given to severely ill patients. The attribution can be perfectly faithful to the predictor while being the opposite of a causal treatment effect. A hiring model may use a geographic feature that correlates with a social variable; changing the recorded geography may change the prediction without changing any underlying qualifications. This is why SHAP documentation itself warns that making predictive correlations transparent does not make them causal.

A practical explanation review can be organized around three tests. The semantics test asks whether the method's output corresponds to the question being asked. The evidence test asks whether the claimed relationship with the model has been validated by intervention, controlled perturbation, formal analysis, or another appropriate experiment. The utility test asks whether the explanation improves the downstream task for its intended audience without creating dangerous overconfidence.

Passing only one test is not sufficient. A beautiful heat map may be useful for communication but fail the evidence test. A mathematically well-defined attribution may be faithful to a model yet irrelevant to a user's decision. A counterfactual may change the classifier output while failing the semantics of real-world recourse. Mature XAI engineering therefore resembles requirements engineering combined with experimental science.

The explanation contract turns that requirement into an interface that can be reviewed, tested, and versioned. With the contract in place, we can ask a more uncomfortable question: if the task can be solved with a model whose structure is directly inspectable, why create avoidable opacity?

1.3 Interpretable models before post-hoc explanations

Interpretable models are often caricatured as linear models that trade accuracy for comfort. That caricature is outdated. The more useful distinction is between unconstrained complexity and structured complexity. A generalized additive model can learn nonlinear response curves. An Explainable Boosting Machine can model flexible univariate effects and selected interactions. Rule lists, monotonic models, sparse scoring systems, prototypes, and concept-based architectures expose different parts of the computation while remaining substantially richer than a textbook linear regression.

The engineering advantage is not that a human can mentally simulate every arithmetic operation. It is that the model's meaningful degrees of freedom are visible. A clinician can inspect whether risk rises smoothly with a laboratory value or whether there is an implausible discontinuity. A credit-risk team can examine whether the learned shape for account age behaves sensibly and whether a particular interaction dominates unexpectedly. When the structure itself is inspectable, many debugging tasks become direct rather than inferential.

A concrete workflow is to treat the interpretable model as the first serious model, not as a token baseline. Tune it properly. Calibrate it. Give it the same feature engineering and validation discipline as the black-box candidate. Compare errors at the level of operational consequences rather than only aggregate accuracy. It is common to discover that the apparent “cost of interpretability” was partly the cost of comparing a carefully optimized deep model with a deliberately weak transparent baseline.

At the same time, visibility can mislead. A response curve may be easy to read and still encode confounding, sample-selection bias, or a proxy for a protected characteristic. Interpretability reduces one epistemic barrier; it does not grant causal validity. The right posture is therefore neither “transparent models are safe” nor “complex models are unavoidable.” It is to use structure as an observability advantage and then test the system with the same seriousness applied to any other high-stakes model.

A useful real-world pattern is the “transparent challenger.” Even when a team expects to deploy a complex model, it can maintain a strong interpretable model alongside it. The challenger serves several roles at once: a performance baseline, a source of global response curves, a detector of unexpected feature behavior, and a fallback when the black

box enters an uncertain regime. Large disagreement between the two models is itself an interpretable event worth investigating.

This pattern also helps separate task difficulty from architectural fashion. If an EBM and a deep tabular model make nearly identical errors, the application may not benefit materially from the extra opacity. If the black box wins specifically on a narrow subgroup or interaction regime, the team has learned where the added complexity buys capability and can concentrate assurance work there. Interpretability then becomes part of model comparison rather than a philosophical preference imposed after the fact.

Cynthia Rudin's influential 2019 argument remains a useful corrective to the assumption that accuracy requires opacity. In high-stakes settings, if a competitive interpretable model can be built, replacing it with a black box and then approximating that black box with a post-hoc explainer creates an unnecessary epistemic gap. The surrogate explanation is not the decision rule; it is another model of the decision rule.

The point is not that every interpretable model is simple in the everyday sense. Modern generalized additive models, sparse tree sums, monotonic models, prototype architectures, and concept-based networks can represent nontrivial relationships while exposing structure that domain experts can inspect. Interpretability is an architectural budget: one chooses which forms of complexity are permitted and which are made visible.

This is especially attractive when the task is tabular and structured, the number of meaningful variables is moderate, the cost of an error is high, and domain experts can evaluate feature-response relationships. It becomes harder when the raw input is high dimensional, such as pixels, waveforms, code, or natural language, because the features presented to the model are themselves far from human concepts.

Generalized additive models decompose a prediction into contributions associated with individual features and, optionally, a controlled number of interactions. The engineering appeal is that each feature can be visualized as a response curve: one can see how age, temperature, account age, or another variable changes the model score while holding the rest of the additive structure fixed. This supports global inspection rather than post-hoc reconstruction.

Explainable Boosting Machines, implemented in InterpretML, use boosted tree components to learn flexible shape functions while retaining the additive form. In many tabular tasks they can compete surprisingly well with black-box alternatives. The important lesson is not that EBMs always win. It is that model selection should compare an interpretable

frontier against an opaque frontier instead of comparing one simple linear baseline with a highly tuned black box and declaring interpretability too expensive.

An interpretable model can still be wrong, biased, unstable, poorly calibrated, or causally confused. A short decision tree can encode a discriminatory policy. A sparse model can rely on a proxy. A concept bottleneck can use mislabeled or incomplete concepts. Intrinsic visibility removes one layer of opacity, but it does not validate the dataset, objective, deployment context, or causal assumptions.

This matters because interpretable models often receive less scrutiny precisely because their logic appears understandable. The appropriate engineering stance is to treat transparency as increased observability. Increased observability makes testing easier; it does not replace testing.

There are tasks for which current transparent models do not provide the same capability. Raw images, speech, open-ended language, code generation, scientific surrogate modeling, and general-purpose multimodal reasoning can require representation learning at a scale that resists direct human inspection. In those cases the black box is not an ethical failure by definition. It is an engineering choice with a bill attached.

That bill has several parts. First comes validation: if the internal logic cannot be directly inspected, stronger behavioral tests are needed across realistic and adversarial conditions. Second comes observability: the system should expose uncertainty, provenance, retrieval sources, tool calls, relevant intermediate states, and model/version identifiers. Third comes explanatory testing: post-hoc methods need sanity checks, stability analysis, and comparison against alternative methods. Fourth comes incident readiness: teams need enough retained evidence to reconstruct why a consequential output occurred after the model, prompt, retrieval index, or data pipeline has changed.

A useful analogy is technical debt. An opaque model can be the right choice in the same way that a complicated distributed system can be the right architecture. But nobody would claim that distributed complexity is free because latency improved. The organization pays through monitoring, debugging, failure analysis, testing, and operational discipline. Opacity should be budgeted the same way. If a black box buys a trivial improvement, the debt may be irrational. If it enables a capability that matters, the added assurance work becomes part of the design rather than an optional compliance layer.

The decision should also be reversible. Teams often lock themselves into opaque systems because the explanatory interface is built around one particular model family. A healthier architecture separates the task

API from the assurance API. This allows an interpretable model, a black box, or an ensemble to be substituted while keeping a stable set of questions: what changed, what evidence supports the output, where is the system uncertain, and what can be reproduced. Explainability then becomes a property of the engineering system rather than a decorative feature of one model.

Consider a document-understanding system that must extract obligations from contracts, reason across many pages, and draft a structured review. A linear or additive model cannot perform the full task. The correct response is not to pretend that a language model can be made transparent by attaching a few token attributions. Instead, the system should be decomposed into observable stages: document ingestion, retrieval, claim extraction, cross-reference resolution, reasoning, and final generation. Each stage can expose its own evidence and failure conditions.

This decomposition illustrates an important idea: architectural transparency can compensate partly for model opacity. The foundation model may remain mechanistically difficult to understand, yet the application can be designed so that claims are linked to source spans, tool calls are logged, intermediate objects are validated, and high-risk actions require explicit checks. The explanatory burden is distributed across the system rather than concentrated in one magical post-hoc explainer.

The same logic applies to autonomous driving, medical imaging, scientific foundation models, and coding agents. When a black box is justified by capability, the surrounding architecture should become more explicit, not less. The question is no longer whether the neural network can narrate its internal thought process. The question is whether the whole system exposes enough independent evidence for humans and automated monitors to detect when the learned component is wrong.

There are legitimate cases for complex opaque models. Raw perception and language tasks may require architectures whose learned representations are far more expressive than manually specified features. Foundation models can deliver capabilities that no compact transparent alternative currently matches. In these settings, the engineering question changes from 'why not use an interpretable model?' to 'which observability and assurance mechanisms compensate for the opacity we are choosing?'

The answer should be proportional to risk. Low-consequence ranking or personalization may require only behavioral monitoring and selected local explanations. Clinical decision support, autonomous control, or

models that can take consequential actions may require stronger validation, independent tests, uncertainty estimation, causal checks, security analysis, and possibly mechanistic or formal evidence for selected properties.

A useful selection protocol evaluates predictive performance, calibration, robustness, interpretability, operational cost, and auditability together. Start with a transparent baseline that is genuinely competitive and properly tuned. Compare it with more complex alternatives on the errors that matter rather than on one aggregate metric. If opacity buys only a marginal gain, the simpler system may have better total engineering value. If opacity buys a material capability, record exactly what new assurance work becomes necessary.

Interpretability does not guarantee correctness, and a readable model can still encode a bad dataset or a harmful objective. Its advantage is narrower and valuable: it reduces the distance between model behavior and the evidence needed to inspect it. When opacity is unavoidable, the rest of the book describes the additional assurance burden that follows.

Chapter 2. Interrogating Predictions

Once the explanatory question has been specified, the next problem is measurement. Attribution methods measure some form of dependence, local surrogates approximate behavior in a chosen neighborhood, counterfactuals describe changes that cross a decision boundary, concept methods lift internal activity into a human vocabulary, and data-attribution methods ask which training examples mattered. These are powerful tools precisely because they answer different questions. They become misleading when their outputs are translated into stronger language than their assumptions permit. This chapter moves from the simplest local measurements to richer contrastive and semantic explanations, repeatedly asking the same engineering question: what was changed, what was observed, and what would make us reject the interpretation?

QUESTION TO CARRY THROUGH THE CHAPTER — What does the method measure, which comparison defines the measurement, and how could the interpretation fail?

2.1 Attribution as measurement, not truth

Attribution begins with an apparently simple desire: divide a prediction into pieces. For a tabular model the pieces may be variables; for an image they may be pixels or regions; for language they may be tokens or embedding dimensions. The difficulty is that a prediction is produced by interactions. Once features are correlated, redundant, discrete, or meaningful only in combinations, there is no unique natural way to assign credit.

A helpful way to teach attribution is through a temperature metaphor. A thermometer is meaningful only because we know what physical quantity it measures and how it is calibrated. An attribution plot should be treated the same way. Gradients measure local sensitivity. Integrated Gradients measure accumulated sensitivity along a path from a baseline. Occlusion measures the effect of replacing or removing information according to a masking rule. Shapley-style values distribute a difference in output across

feature coalitions relative to a background. These are related but not interchangeable quantities.

The baseline problem deserves special attention because it hides inside many attractive visualizations. When a medical image is compared with an all-black image, the reference is not “nothing”; it is a particular pixel array that may be far outside the training distribution. When a token is removed from a sentence, the masked sequence may be ungrammatical. When a financial variable is set to zero, the method may have transformed “unknown” into “literally zero.” A baseline is a counterfactual world. Engineers should choose it deliberately and test whether reasonable alternative baselines tell materially different stories.

For images and long sequences, attribution also creates a resolution problem. A heat map can be faithful at the pixel or token level and still be unusable to a person who thinks in objects, findings, clauses, or events. This is one reason the field is moving toward concepts and semantic representations. Low-level attribution remains valuable, especially for debugging and controlled tests, but a mature explanation stack often needs a mapping from computational units to domain units rather than assuming they are the same.

Figure 3.1. Attribution families answer related but different operational questions.

An attribution method produces a score for each input feature, token, pixel, region, or embedding dimension. The score is often visualized as importance, but ‘importance’ is not a primitive of nature. It must be defined through a reference, a perturbation, a derivative, a coalition game, or another operational rule. Two methods can disagree because they estimate different quantities rather than because one implementation is broken.

This is clearest in the presence of correlated or redundant variables. If two features encode almost the same information, removing one may barely change the prediction because the other substitutes for it. A gradient can be small because the model is locally saturated even when the feature was essential to reaching the current state. An attribution can change when the background distribution changes because ‘missing’ features are being replaced by a different counterfactual world.

The simplest differentiable attribution asks how the output changes under an infinitesimal change to the input. Gradients are cheap and directly connected to the local geometry of a neural network. In images, the absolute gradient can be displayed as a saliency map; in text, gradients can be aggregated over token embeddings; in structured neural models, gradients can expose local sensitivity.

The limitation is semantic. A gradient measures local sensitivity, not historical contribution, necessity, or causal effect. Saturated nonlinearities can suppress gradients. High-frequency changes can create noisy maps. The choice of output logit, probability, loss, or other target changes the result. For discrete features such as tokens, the derivative lives in embedding space rather than directly in the space of valid words.

The landmark sanity-check work of Adebayo and colleagues demonstrated why a visually plausible saliency map is weak evidence. Some methods produced similar-looking maps even after model parameters or labels were randomized. The conclusion is not that saliency is useless. It is that an explainer claiming to describe a trained model should usually respond when the trained model is destroyed.

Integrated Gradients was designed around explicit axioms, especially sensitivity and implementation invariance. Instead of using only the derivative at the observed input, it accumulates gradients along a path from a reference baseline to the input. The resulting attribution answers a more specific question: how did the model output change as the input moved from the chosen reference state to the observed state along this path?

The baseline is therefore part of the explanation. A black image, a blurred image, a zero vector, a padding token, an average embedding, or a domain-specific reference can produce materially different semantics. There is no universally neutral baseline. In medical imaging, zero intensity may or may not represent absence of signal. In language, a zero embedding is not necessarily a valid sentence. In finance, zero income describes a particular person, not missing income.

For engineering use, baseline sensitivity should be measured rather than hidden. If reasonable baselines produce contradictory narratives, the explanation should be considered underdetermined. Captum exposes Integrated Gradients and related attribution methods in PyTorch, including workflows for transformer and LLM inputs, but the library cannot decide what baseline is scientifically meaningful for the application.

Methods such as Grad-CAM explain convolutional or vision systems by combining activation maps with gradient information to produce spatial localization. They are effective communication tools because a coarse heat map over an image maps naturally onto how humans inspect scenes. The same intuitive power creates risk: smooth colorful regions look like evidence even when the method is only a rough localization heuristic.

For modern vision transformers and multimodal models, attribution can target patches, attention heads, latent tokens, cross-modal links, or

internal features. The number of plausible explanatory units increases dramatically. Engineers should therefore distinguish the unit being scored from the semantic object a user sees. A patch attribution is not automatically an object attribution; an attention-head attribution is not automatically a concept attribution.

Perturbation methods estimate importance by changing or removing features and observing output changes. This gives them a direct behavioral interpretation, but only relative to the perturbation procedure. Replacing a pixel with zero, masking a word, shuffling a column, or sampling a value from a background distribution may create inputs the model never encountered during training.

Off-distribution perturbations are one of the central confounds in XAI evaluation. A dramatic output change may reflect the model's reaction to an unrealistic input rather than the importance of the removed feature. Conditional perturbations and generative in-painting can produce more realistic alternatives, but then the explainer inherits assumptions and errors from the conditional or generative model.

Attribution therefore gives us measurements, not final causes. The next family of methods goes one step further by constructing local approximations and allocation schemes, which are useful only if we keep their neighborhoods and background assumptions visible.

2.2 Local surrogates and Shapley explanations under assumptions

LIME and SHAP became popular because they transformed the abstract problem of black-box explanation into something operational. LIME fits a simple model around one instance. SHAP frames attribution as a cooperative allocation problem with a strong axiomatic foundation. Both can be extremely useful. Neither exempts the engineer from specifying the neighborhood, reference distribution, masking semantics, or feature dependence assumptions that make the result meaningful.

Consider a fraud model in which merchant category, transaction amount, time of day, device identity, and recent account activity are strongly dependent. If a model-agnostic explainer perturbs each feature independently, it may generate synthetic examples that resemble no legitimate transaction. The local surrogate can then be accurate on the artificial neighborhood while poorly representing the model on the real data manifold. A SHAP implementation that treats features as independent can likewise distribute credit differently from one using a

conditional background. The resulting bar chart does not display a context-free truth; it displays the answer under the chosen game.

Validation should therefore be built into the workflow. Recompute explanations under multiple sensible backgrounds. Randomize model parameters or labels when the method claims sensitivity to learned structure. Perturb the features ranked as important and test whether predictions respond as expected, while checking that the perturbations remain meaningful. Compare methods that operationalize different notions of importance. Inspect subgroups and boundary cases rather than only median examples. If a method can be adversarially gamed, treat the explanation channel as an attack surface rather than as a trusted telemetry feed.

The practical goal is not to find a single “best explainer.” It is to establish a chain of evidence around a particular claim. If a feature is said to be important, the team should be able to state what important means, show that the result depends on the trained model, show that it is not an artifact of one arbitrary baseline, and describe the circumstances under which the conclusion becomes unstable. That discipline turns attribution from a visual accessory into an instrument.

Before accepting an attribution, test whether it changes when the model changes, whether it is stable under irrelevant transformations, whether different reasonable references change the conclusion, and whether removing allegedly important information alters behavior in the predicted direction. Compare against simple baselines, including random rankings. If the explanation will be used for science or policy, introduce controlled synthetic cases where the relevant mechanism is known.

LIME explains a prediction by sampling perturbed versions of the input, querying the black-box model, weighting the samples by proximity to the instance of interest, and fitting a simple interpretable surrogate. The method is attractive because it needs only a prediction interface. The target model can be a neural network, random forest, remote API, or any other system that returns outputs for inputs.

The local surrogate is not the original model. Its fidelity depends on how the neighborhood is generated, how distance is defined, how samples are weighted, how features are represented, and how the surrogate is regularized. Two LIME runs can differ because the sampling process is stochastic. More importantly, they can differ because there is no unique local linear approximation to a complex decision surface at a finite neighborhood scale.

This makes LIME useful as an engineering probe rather than as a certificate. Repeated runs, alternative neighborhood radii, realistic

perturbation schemes, and direct measurement of local surrogate fidelity should be standard. A single screenshot is not a validation protocol.

Anchors replace a local linear story with high-precision if-then conditions: when a set of feature conditions holds, the model's prediction remains stable with high probability under a defined perturbation distribution. For many users, a short rule is more understandable than a vector of coefficients. The price is that the rule can cover only a narrow region and its precision is again defined relative to how alternative inputs are sampled.

Anchors illustrate a broader lesson: explanation form changes what can be claimed. A sparse rule can express a sufficient condition in a region, but it does not necessarily identify every influential feature. A coefficient vector can rank local effects, but it does not necessarily define a stable decision region. Choosing an explanatory representation is therefore part of the problem specification.

SHAP brought cooperative-game ideas into mainstream model explanation. A prediction is treated as a payoff and features participate in coalitions. The explanatory task is to allocate the difference between a reference prediction and the observed prediction among the features. Under particular axioms, Shapley-style values provide a principled additive allocation.

This foundation is powerful because it clarifies desirable consistency properties and creates a common interface across model families. TreeSHAP makes Shapley-style explanations practical for tree ensembles; other explainers approximate the coalition calculation for general models. Aggregating local SHAP values produces useful global summaries and interaction views.

Yet a Shapley value is only as meaningful as the coalition game. What does it mean for a feature to be missing? Should missing values be sampled independently, conditionally on observed features, or according to a causal intervention? How should dependent variables be handled? Which background dataset defines the reference? Different answers induce different explanations while all remaining mathematically legitimate.

Correlation is not a minor edge case. Real datasets are full of derived variables, proxies, repeated measurements, and strongly coupled attributes. If a person's occupation and income are correlated, assigning credit independently can create unrealistic coalitions. Conditional approaches preserve statistical dependence but can allocate credit to a feature that the model never reads because the feature carries

information about another input. Interventional approaches more closely reflect the model function but can create unrealistic combinations.

There is no universal resolution because the disagreement reflects different explanatory questions. One question asks how the model function depends on each input when other inputs are manipulated independently. Another asks how much information a feature contributes given the observational data distribution. A third asks about causal interventions in the world. Engineers must label which question is being answered.

Post-hoc explainers can themselves become attack surfaces. Slack and colleagues showed that perturbation-based methods such as LIME and SHAP can be fooled by models that detect the artificial perturbations generated by the explainer. The model behaves one way on normal inputs and another on the synthetic neighborhood, causing the explainer to report a benign surrogate for a problematic decision mechanism.

This result is important beyond one attack. Any explainer that relies on a predictable query distribution can be gamed by a strategic model or wrapper. In adversarial or regulated contexts, explanations should not be generated by a process that the target system can trivially identify and special-case. Independent behavioral audits and randomized challenge distributions are needed.

Use these methods when their explanatory semantics match the question and when black-box access makes more direct instrumentation unavailable. Record the background dataset, masking strategy, output target, random seed, explainer parameters, model version, and data version with every explanation. For consequential uses, run sensitivity analysis across plausible backgrounds and compare multiple independent explanation families.

Local surrogates and Shapley explanations are strongest when treated as explicitly parameterized experiments. Once the question changes from dependence to change—what would have to be different for the outcome to change—we enter the counterfactual problem.

2.3 Counterfactuals, recourse, and causal structure

Attribution asks which existing inputs matter under a specified comparison. Counterfactual explanation changes the grammar of the question: what would have to be different for the result to change?

Counterfactual explanations start from a human pattern of reasoning: “Why did this happen rather than that?” A loan applicant does not

necessarily care that income received 0.27 units of attribution. They care whether a realistically higher income, a smaller requested amount, or a longer credit history would have changed the decision. A physician may ask which observable change would have moved a patient out of a high-risk category. An operator may ask what input condition would have prevented an automated shutdown. The explanation is contrastive because it is defined against another outcome.

This makes counterfactuals immediately intuitive and immediately dangerous. Optimization algorithms are excellent at finding small changes in input space. They are not automatically good at finding possible changes in the world. A nearest counterfactual for a hiring model might change age, an immutable attribute, or alter educational history in a way that cannot be achieved prospectively. A clinical counterfactual might lower a measured biomarker while ignoring the disease process that determines it. A text classifier might be flipped by inserting a nonsense token that no human would regard as a meaningful revision.

The move from counterfactual explanation to recourse changes the objective. Recourse asks for a feasible path from the current state to a state associated with a desirable outcome. That path has costs, constraints, dependencies, time, and uncertainty. Some features are immutable; some can be influenced only indirectly; some changes affect other variables; some actions that alter the model score do not alter the true objective. A recourse system therefore needs domain knowledge that is not contained in the predictor.

The most useful design pattern is to separate three layers. The predictor states how the system maps current observations to an outcome. The feasibility model states which changes are allowed and how variables depend on one another. The policy layer states which actions are acceptable to recommend. By keeping these layers distinct, an engineer can explain why an optimization is mathematically valid yet socially or physically invalid, rather than forcing all semantics into one distance function.

An engineering scenario makes the distinction concrete. Suppose an energy-management model recommends shutting down a machine because predicted failure risk exceeds a threshold. A nearest counterfactual might say that lowering vibration sensor 7 by five percent would avoid shutdown. That is informative for debugging the classifier but useless as recourse if sensor 7 cannot be changed directly. A feasible recourse analysis might instead identify bearing replacement, load reduction, or cooling changes that are expected to alter several sensor readings together. The explanatory object has moved from a modified vector to an action sequence.

This is why good counterfactual systems often need constraints expressed in domain language. Some changes are one-way, some have costs, some require time, and some are mutually dependent. Encoding those constraints is not a nuisance around the algorithm; it is where the real semantics of recourse live.

People often explain decisions contrastively: why this outcome rather than another? Counterfactual explanations operationalize that intuition by searching for an alternative input close to the observed one that changes the model's decision. A rejected applicant might receive an explanation such as: if annual income had been higher while the other modeled features remained within specified values, the classifier would have approved the application.

This table separates four objects that are frequently presented under the same counterfactual label. The distinctions matter because the burden of evidence increases each time the explanation moves closer to advice.

Table 2.1 — A counterfactual becomes stronger as it incorporates constraints about the world.

Concept / object	Engineering meaning
Classifier counterfactual	A changed input that crosses a decision boundary. It says what would change the model output, not whether the change is possible.
Feasible counterfactual	A boundary-crossing input that respects domain constraints and immutable variables.
Actionable recourse	A feasible change expressed as an intervention a person or system can realistically execute.
Causal recourse	An intervention justified by assumptions about how changing one variable propagates to others in the underlying system.

A user-facing system should therefore state which level it actually supports. A classifier counterfactual can be valuable for debugging; presenting it as causal recourse without the necessary model of feasible intervention would be an unjustified semantic upgrade.

The appeal is immediate. Unlike a feature-attribution vector, a counterfactual can connect directly to an alternative outcome. It also avoids revealing the full decision boundary. Wachter, Mittelstadt, and Russell helped establish this framing for automated decisions,

emphasizing that a counterfactual can support understanding and contestability without exposing the internal model.

Most naive counterfactual generation begins by minimizing a distance between the original instance and an alternative that crosses the decision boundary. But geometric proximity is only a weak proxy for human action. Age cannot be decreased, a university degree cannot be edited instantaneously, a medical measurement may be a consequence rather than a cause, and changing one variable may necessarily change others.

Feasibility constraints improve the search by marking immutable variables, imposing valid ranges, preserving categorical relationships, and accounting for action costs. Diversity is also useful because a single nearest counterfactual may not reflect the alternatives available to a person. DiCE and related approaches generate multiple alternatives to expose different possible routes.

Even a feasible point can be a poor recommendation. A person's observed state is the result of a process. If income, education, occupation, and location are causally related, directly replacing one number in the feature vector does not describe the consequences of a real intervention.

Algorithmic recourse treats the explanation not merely as a description of the classifier but as a recommendation that should improve an individual's outcome. Karimi, Schölkopf, and Valera make the distinction explicit: a counterfactual endpoint and a causal intervention are not the same object. Recourse requires modeling which variables can be changed, how interventions propagate through the system, and whether the favorable outcome persists under uncertainty.

This distinction has major engineering consequences. A credit model may tell a person that reducing utilization would change the score. If utilization is itself constrained by income shocks and fixed expenses, a technically valid classifier counterfactual may be practically inaccessible. A medical risk model may suggest altering a biomarker that is not directly controllable. A fraud detector may reveal how to evade detection rather than how to become non-fraudulent.

Causal reasoning enters when the question changes from “what input alteration flips the model?” to “what intervention could change the underlying outcome?” The difference is easiest to see in medicine. Suppose treatment is strongly associated with mortality because the sickest patients receive it. A predictive model may correctly use treatment as a marker of severity. An attribution method may faithfully report that dependence. But recommending “do not treat” from that attribution would

reverse the causal meaning. The model has learned an observational pattern, not a treatment effect.

Structural causal models give a language for representing such dependencies, but they do not remove uncertainty. The graph itself is an assumption, often informed by domain knowledge and imperfect data. Unobserved confounding can invalidate seemingly precise results. The correct engineering lesson is not that every XAI system must become a full causal-inference project. It is that action-oriented explanations should explicitly state when they rely only on predictive dependence and when they rely on a causal model.

Robust recourse adds another practical complication: models change. A user may follow a recommendation generated today only to encounter a retrained model tomorrow. If a suggested action lies barely across the current decision boundary, small model updates can erase the promised benefit. Research on robust recourse therefore seeks actions likely to remain effective across plausible model uncertainty or retraining. This is especially important when actions are expensive or slow, such as education, debt reduction, or clinical intervention.

A good recourse interface should therefore communicate more than one optimized point. It should communicate feasible ranges, assumptions, uncertainty, and the distinction between guaranteed system behavior and expected real-world effect. The closer an explanation moves toward advice, the less acceptable it is to hide these assumptions behind a neat sentence saying “change X by 12 percent.”

Causal models are especially valuable when the recommendation itself can alter future data. In credit, granting a loan changes debt, repayment history, and later eligibility. In medicine, treatment changes biomarkers that the prediction model will observe in subsequent visits. In industrial control, an intervention changes the state trajectory of the machine. A static counterfactual can ignore these feedback loops and therefore recommend an action whose short-term classifier effect differs from its long-term system effect.

For this reason, robust recourse can be treated as planning under model uncertainty. Instead of optimizing one boundary crossing, evaluate a family of plausible predictors, causal parameters, or future states. Prefer actions whose benefit is stable across that family and whose cost remains acceptable if the world deviates from the nominal model. When no such action exists, the system should say so rather than manufacture precision.

The user-facing explanation should separate what is guaranteed from what is estimated. “This change flips the current classifier” is a different statement from “this action is expected to improve the underlying

outcome.” Mixing them is one of the easiest ways for technically correct XAI to become practically misleading.

Causal explanation methods try to incorporate structural knowledge into attribution. Causal Shapley values, for example, use a causal graph to separate associations from intervention effects and can distinguish direct from indirect contributions. The gain is stronger semantics. The cost is stronger assumptions: causal graph structure, intervention definitions, and identifiability become part of the explanation system.

This is not a defect. All explanations have assumptions; causal methods make more of them explicit. The danger is to hide uncertain causal structure behind mathematically sophisticated outputs. If the graph is wrong, the causal explanation can be confidently wrong. Engineering practice should therefore version causal assumptions, record which edges are expert-provided versus learned, and test sensitivity to plausible alternative graphs.

A further practical problem is model drift. If a person invests time or money in following a recommendation, the model may be retrained before the action is completed. Robust recourse seeks interventions that remain effective across plausible model updates or uncertainty sets. This turns explanation into a contract across time rather than a statement about one model snapshot.

For production systems, counterfactual generation should therefore be connected to model lifecycle management. A recourse artifact should carry the model version, validity horizon, assumptions, and confidence. If the policy or model changes, stale recommendations should be detectable.

The boundary between counterfactual explanation and recourse is where model semantics meet world semantics. That same transition motivates explanations that use concepts, examples, and training data rather than raw input coordinates.

2.4 Concepts, prototypes, and the data behind a decision

Human explanations rarely stop at elementary features. A radiologist thinks in lesions, borders, anatomical regions, and diagnostic patterns; a security analyst thinks in tactics, behaviors, and sequences; a software engineer thinks in functions, invariants, and failure modes. Concept-based XAI tries to align the explanatory vocabulary with these units. The attraction is obvious: a statement that a vision model depends on “striped

texture” or “wheel-like geometry” is easier to inspect than thousands of pixel scores.

TCAV provides one route by defining a concept from examples and asking whether the model’s internal representation is sensitive in the concept direction. Concept bottleneck models go further by making intermediate concepts part of the predictive architecture. Prototype systems explain by similarity to learned or selected examples, giving a “this looks like that” form of reasoning. These approaches differ in whether concepts are post-hoc probes, explicit bottlenecks, or case-based structures, but all expose the same fundamental issue: the quality of the explanation now depends on the quality of the concept vocabulary.

Concepts can be incomplete, culturally contingent, mislabeled, or too coarse. A concept bottleneck may create the comforting appearance that the model reasons only through approved concepts while hidden paths or imperfect concept annotations preserve unwanted information. A prototype may be visually compelling yet represent a spurious similarity. Editing a concept can improve controllability while also introducing distribution shift. Concept-based explanation therefore exchanges low-level opacity for a new modeling problem: defining, learning, and validating the semantic units themselves.

This is still a productive trade. It is often easier for experts to challenge a wrong concept than a wrong latent vector. The strongest systems treat concepts as hypotheses that can be revised rather than as sacred labels. They expose examples that activate a concept, counterexamples that should not, uncertainty in concept prediction, and downstream sensitivity to concept edits. In that form, concepts become a collaborative interface between learned representations and domain expertise.

Input features are convenient because models consume them directly, but they are often poor explanatory units. A radiologist reasons in terms of lesions, borders, asymmetry, tissue patterns, and diagnostic criteria rather than individual pixels. A software engineer reasons about API misuse, privilege boundaries, race conditions, and protocol states rather than token IDs. Concept-based XAI attempts to bridge this semantic gap.

The hard part is that concepts are not given by the network. They may be human-defined, discovered from representations, inherited from an ontology, or generated by another foundation model. Every choice embeds a worldview into the explanation. Concept quality therefore deserves the same scrutiny as model quality.

Testing with Concept Activation Vectors, or TCAV, asks whether a model’s internal representation is sensitive to a user-defined concept. Example data for a concept such as stripes, texture, gender presentation,

or a medical pattern are used to estimate a direction in representation space. The model's output sensitivity along that direction is then measured.

TCAV is valuable because it lets domain experts ask questions in their own vocabulary without retraining the entire predictor. It is also limited by the concept examples, the choice of layer, linear separability, and the assumption that a direction corresponds to the concept in a sufficiently stable way. A concept probe can detect information that is represented without proving that the model causally uses that information for a specific prediction.

Concept Bottleneck Models make a stronger architectural commitment. Instead of merely probing hidden representations after training, the model explicitly predicts named concepts and then predicts the final target from those concepts. This creates a human-readable intermediate interface and supports intervention: a domain expert can correct a concept and observe how the final output changes.

The original work by Koh and colleagues showed that concept bottlenecks could achieve competitive performance in tasks such as bird identification and radiographic grading while enabling concept-level inspection. Later work introduced probabilistic concept bottlenecks, editable concept systems, and more data-efficient concept learning. The frontier is moving from static concept dictionaries toward representations that can be revised, queried, and aligned with richer semantic spaces.

The central unresolved issue is concept completeness. If the model can only use a fixed vocabulary, missing concepts can reduce accuracy or force inappropriate compression. If a residual hidden channel is added to recover lost information, the system may regain performance while losing the guarantee that the visible concepts fully mediate the decision. An engineer should therefore ask not only whether concepts are understandable but whether they are sufficient.

Prototype models explain by similarity: this input, or this part of an input, resembles a learned representative example. ProtoPNet popularized an architecture for image classification in which local image regions are compared with learned prototypical parts. The explanation has a form humans use naturally: 'this looks like that.'

Prototype quality is not automatic. A learned prototype can be visually ambiguous, unrepresentative of the class, or semantically aligned with a shortcut. A prototype can also encourage a user to overgeneralize from one case. Prototype systems need audits of representativeness, coverage, stability, and class semantics, especially when the examples contain sensitive or private information.

A model is partly a compressed history of its training data. When a surprising prediction appears, asking which training examples most influenced it can be more informative than asking only which input features mattered. Influence functions, representer-style methods, nearest-neighbor analyses in learned spaces, gradient similarity, and newer data-attribution techniques all attempt to trace behavior backward toward the examples or groups that shaped it.

This is valuable for debugging label errors, identifying duplicates, finding contamination, understanding memorization, and investigating whether a behavior is inherited from a narrow slice of data. In a language model, the problem becomes much harder because training corpora are enormous and the relationship between one output and individual examples is diffuse. Even so, the conceptual move is important: explanation can concern not only the forward computation of a model but also the provenance of the knowledge embedded in it.

Training-data attribution also reveals why semantic abstraction is a frontier problem. Raw influential examples are often too numerous or too specific. Engineers need to summarize them into patterns without inventing a story that the evidence does not support. A collection of influential cases might reveal a recurring template, domain, source, demographic cluster, or annotation artifact. Turning that cluster into a named concept is a second inference step and should be validated separately.

In practice, a useful data-centric explanation often combines levels. It shows representative influential examples, the metadata or provenance associated with them, the common pattern inferred from the group, and an intervention test in which removing or reweighting the suspected data changes the behavior. This creates a stronger narrative than simply returning the nearest training item. It also points toward a future in which explanation systems connect model behavior to lineage graphs covering data, transformations, training runs, and evaluation evidence.

Data attribution becomes particularly important in generative systems because failures often look like isolated output mistakes while their causes are historical. A model that repeatedly reproduces an obsolete legal phrase, a biased association, or a distinctive memorized passage may have learned the behavior from a narrow set of examples. Identifying those examples can support remediation: deduplication, relabeling, reweighting, targeted fine-tuning, or retrieval controls.

The difficult part is scale. In modern training pipelines there may be no single influential example. Thousands of related items can collectively shape a representation. A useful engineering abstraction is therefore

hierarchical provenance: individual examples at the bottom, clusters or sources in the middle, and semantic patterns at the top. Investigations can begin with a model behavior and move backward until they reach a level at which an actionable data change can be made.

This is also where privacy and explanation collide. The more directly an explanation exposes training examples, the more carefully access must be governed. Internal debugging may justify exact records under controlled access, while external explanations may need aggregation. Explainability does not override data-protection obligations; it creates another reason to design provenance systems deliberately.

Feature attribution asks which parts of the current input matter. Training-data attribution asks which training examples shaped the prediction or behavior. Influence functions approximate how upweighting or removing a training example would change parameters, loss, or a target prediction. This can help identify mislabeled examples, duplicated data, poisoned records, or influential cases.

The classic influence-function approach assumes approximations that become difficult for large non-convex networks. Modern data-attribution research therefore includes representer points, gradient-similarity methods, tracing through training checkpoints, and scalable approximations. For foundation models, the problem becomes formidable because the training corpus is enormous, training is multi-stage, and behavior can arise from distributed statistical regularities rather than one memorized example.

Nevertheless, data provenance is increasingly important. If a model produces a copyrighted passage, unsafe behavior, or a scientific claim, engineers and auditors may want evidence connecting the behavior to training or retrieval sources. Training-data attribution and retrieval provenance are distinct; both need explicit support in future assurance systems.

The deeper challenge behind all these methods is semantic abstraction. Neural networks operate in distributed high-dimensional representations, while humans reason with concepts that are discrete, hierarchical, contextual, and culturally shaped. A single neuron may be polysemantic; a concept may be represented across many components; the same representation direction may acquire different meanings in different contexts.

Recent mechanistic work using sparse autoencoders, semantic embedding spaces, and natural-language activation descriptions can be seen as attempts to build a more scalable bridge between machine representations and human concepts. These approaches are promising

precisely because they shift explanation from raw units toward semantic objects, but the bridge itself is learned and must be validated.

These semantic forms of explanation can be more natural to experts, but they are not exempt from validation. A concept name, prototype, or influential training case remains a hypothesis about what the model uses. Chapter 3 develops the tests needed before any of these artifacts should guide people or policy.

Chapter 3. Testing Explanations in the Real World

An explanation method that looks sensible on one example is not yet an engineering capability. It must survive controlled tests, user interaction, model updates, adversarial pressure, privacy constraints, and the institutional context in which people will act on it. This chapter changes the unit of analysis from the explanation artifact to the explanation system. It begins with faithfulness and falsification, then turns to the human interface and finally to production assurance. The narrative is deliberate: a human-centered explanation that has not been shown to track the model can merely make an error more persuasive, while a faithful explanation that cannot be reproduced after deployment is not an auditable artifact.

QUESTION TO CARRY THROUGH THE CHAPTER — What test would make us reject this explanation, and can the organization reproduce that test after deployment?

3.1 Faithfulness, falsification, and controlled evaluation

XAI becomes scientifically interesting at the moment an explanation is allowed to fail.

The seductive failure mode in XAI is the explanation that looks right. A heat map highlights the object, a rationale uses familiar domain language, a prototype resembles the input, and the reader feels that the system has revealed itself. But visual plausibility is compatible with weak dependence on the trained model. Adebayo et al.'s saliency sanity checks became influential because they demonstrated this point dramatically: some explanation maps changed surprisingly little when model parameters or labels were randomized.

Evaluation therefore begins with a falsification mindset. Ask what should happen if the explanation is genuinely sensitive to the property it claims to measure. If the model is randomized, should the explanation change? If important features are removed, should confidence fall? If a concept is causally relevant, does intervening on its representation

change the output? If a local surrogate is faithful, how rapidly does its accuracy deteriorate as we move away from the explained point? A good metric is not merely a score; it is a test implied by the semantics of the explanation.

Controlled benchmarks help because they can provide ground truth that real-world data rarely supplies. Synthetic or semi-synthetic datasets can specify which features, object parts, rationales, or generative factors are truly relevant. Benchmarks such as CLEVR-XAI, FunnyBirds, ERASER, and OpenXAI expose different aspects of explanation quality. Tools such as Quantus organize a broad collection of faithfulness, robustness, localization, complexity, randomization, and axiomatic metrics. None of these eliminates judgment, because metrics themselves contain perturbation choices and assumptions, but they make evaluation repeatable.

A mature test plan should include negative controls. Explanations should fail when the underlying explanatory relationship has been destroyed. They should also be challenged with distribution shift, correlated features, adversarial inputs, and alternative baselines. The best sign of progress in XAI is not that explanations become prettier; it is that teams can state in advance what evidence would cause them to reject an explanation method for a particular use.

The figure places the explanation artifact in the middle of an experiment. The claim comes first; an intervention then tests whether the model behaves as the claim predicts. The result either strengthens the claim, narrows its scope, or forces its rejection.

Explanation as an experiment

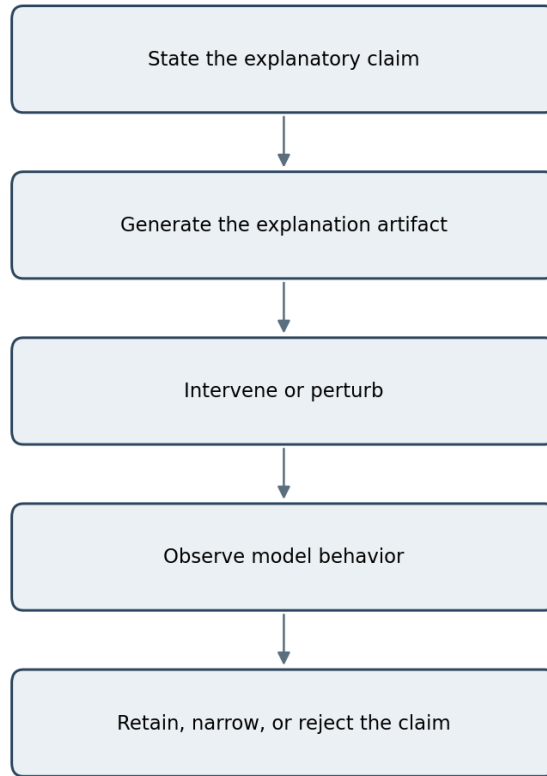


Figure 3.1 — A useful explanation participates in a falsification loop rather than ending the analysis.

This loop is intentionally repeatable. A new model checkpoint, dataset shift, or deployment context can trigger the same tests again. In that sense explanation validation should resemble continuous integration more than a one-time visual inspection.

Figure 6.1. Treat an explanation as an experimental hypothesis, not a finished picture.

For standard supervised learning, labels often provide an external target. For XAI, there is usually no observable ground-truth explanation of a complex trained model. Human rationales are not automatically ground truth because humans and models may solve the task using different cues. A model can be correct for the wrong reason, or wrong for a reason a human would never consider.

This creates an epistemic loop. A method can produce an appealing explanation, but appeal does not establish correspondence with the model. Evaluation therefore relies on controlled interventions, synthetic tasks, randomization, removal experiments, model comparisons, and human studies designed around specific use cases.

The table is best read as an assurance checklist rather than a scorecard. No single metric covers these dimensions, and the relevant subset depends on the explanation contract.

Table 3.1 — Explanation evaluation is multi-dimensional because different tests attack different failure modes.

Concept / object	Engineering meaning
Faithfulness	Does the artifact track the model property it claims to represent?
Robustness	Does the result remain coherent under changes that should be irrelevant to the claim?
Human utility	Does the explanation improve the intended task and calibrate reliance rather than simply increase trust?
Security and privacy	Does the explanation channel leak sensitive information, expose attack surfaces, or enable gaming?
Governance	Can the result be reproduced, versioned, retained, and connected to the model and data that produced it?

For high-stakes uses, a strong result in one column cannot compensate for failure in another. A faithful explanation that leaks training data is not acceptable; a stable explanation that persuades users to over-trust the system is not successful human-centered XAI.

Faithfulness asks whether an explanation reflects the model behavior or mechanism it claims to represent. Plausibility asks whether a human finds the explanation sensible. These dimensions can correlate, but they can also diverge sharply. An unfaithful explanation can be more persuasive because it tells the story a user expects.

This distinction becomes critical with language models because fluent rationales can be generated on demand. A model can produce a convincing chain of reasoning after the decision has effectively been determined by another cue. Human agreement with the rationale measures communicative plausibility, not necessarily causal faithfulness.

Adebayo and colleagues introduced an elegantly simple question: does the saliency method depend on the learned model and data? They progressively randomized network parameters and labels and examined whether explanations changed. Some visually compelling methods were surprisingly insensitive. The test does not establish that a method is faithful when it passes, but failing it is a strong warning.

Sanity checks should be generalized beyond saliency. If a concept explainer claims to describe model internals, randomizing or swapping the relevant components should matter. If a training-data attribution method claims a point is influential, retraining without that point should move the output in the predicted direction. If a mechanistic circuit claims a path is necessary, ablating or patching the path should alter behavior.

A common faithfulness test removes features in order of attributed importance and measures the change in output or accuracy. If the top-ranked features really matter, removing them should hurt more than removing low-ranked or random features. The problem is that deletion itself can create unnatural inputs.

ROAR, or Remove And Retrain, addresses part of this confound by removing features from the training data and retraining the model. Hooker and colleagues showed that comparative conclusions about attribution methods can change under this stronger protocol. The broader lesson is that explanation metrics can rank methods according to artifacts of the evaluation procedure.

Yeh and colleagues formalized infidelity as disagreement between an explanation and the model-output changes induced by perturbations, and sensitivity as how much an explanation changes around nearby inputs. These ideas are useful because they turn intuitive desiderata into measurable tests. Yet no single metric is sufficient.

An explanation can be perfectly stable because it is constant and uninformative. It can be highly sensitive because the actual decision boundary is sharp. Robustness can conflict with fidelity when the model itself is brittle. Sparsity can improve readability while omitting interacting features. Every metric creates incentives, and explanations optimized for one score can game the score.

Controlled benchmarks help because they construct tasks where some aspects of the relevant mechanism are known. CLEVR-XAI uses synthetic visual scenes and question answering to provide spatial ground truth. FunnyBirds creates part-based synthetic images that permit semantic interventions on object parts. ERASER provides human rationales for NLP tasks while separating rationale agreement from faithfulness-oriented

criteria. OpenXAI and Quantus organize multiple explainers and metrics into reproducible evaluation frameworks.

The most important intellectual move in XAI evaluation is to stop asking whether an explanation is convincing and start asking what observation would count against it. This is the difference between illustration and experiment. If a heat map is claimed to identify pixels that the classifier relies on, removing or replacing those pixels should change behavior in a predictable way, subject to the distributional caveats of the perturbation. If a concept score is claimed to identify a representation that causally supports a decision, intervening on that representation should alter the relevant behavior more than matched controls. If no plausible result can make the claim weaker, the artifact is functioning rhetorically rather than scientifically.

This falsification mindset also clarifies why explanation metrics disagree. Deletion scores, insertion scores, infidelity, stability, localization, human preference, and model-randomization checks are not noisy estimators of one hidden scalar called 'explanation quality.' They test different failure modes. Treating them as interchangeable encourages benchmark optimization rather than understanding. A sensible evaluation suite begins with the claim, maps each important assumption to one or more tests, and accepts that some claims require evidence from several methods rather than a single score.

Synthetic ground truth is not a perfect substitute for real applications. A method can excel on toy semantics and fail in a complex domain. But synthetic benchmarks are invaluable for falsification: if a method cannot recover a mechanism in a controlled setting where the answer is known, its interpretation in an uncontrolled setting should be treated cautiously.

No single number can summarize explanation quality because explanations serve different purposes. Faithfulness asks whether the artifact reflects the model. Robustness asks whether it changes reasonably under small, irrelevant perturbations. Localization asks whether it focuses on known relevant regions. Complexity asks whether a human can process it. Stability asks whether similar cases receive compatible explanations. Human usefulness asks whether the explanation improves a downstream task. Privacy and security ask whether the explanation leaks information or creates a new attack surface. These objectives can conflict.

The correct engineering response is to treat explanation evaluation as a test suite rather than a leaderboard. For a debugging explainer, faithfulness and sensitivity to model changes may dominate. For a user-facing recourse system, feasibility, calibration, and comprehension

matter more. For a regulatory audit, reproducibility, subgroup consistency, provenance, and retention of evidence may be central. The scorecard should be derived from the explanation contract rather than copied from the latest benchmark paper.

Human evaluation needs similar discipline. Asking participants whether they “like” an explanation is weak evidence. Better studies test whether users make more accurate decisions, detect model errors, calibrate reliance, identify uncertainty, or complete a task faster without sacrificing correctness. Explanations can increase trust even when trust should decrease. A persuasive but unfaithful rationale may therefore score well on satisfaction while damaging oversight.

The 2026 literature increasingly reflects this broader view. Haufe et al. argue for formal explanatory problem definitions and correctness criteria. Reviews of empirical XAI studies emphasize the heterogeneity of evaluation designs. The practical convergence is toward triangulation: combine model-grounded tests, controlled datasets, user studies, and domain-specific criteria. The objective is not to prove that an explainer is “good.” It is to show that it is adequate for a specified role, under specified conditions, with known failure modes.

A useful way to operationalize multi-objective evaluation is to build an explanation test matrix. Rows represent the claims the organization intends to make: local feature dependence, recourse feasibility, concept use, source reliance, mechanistic relevance. Columns represent evidence dimensions: model sensitivity, robustness, subgroup consistency, human utility, privacy, security, and reproducibility. Empty cells are visible technical debt. The matrix also prevents one strong metric from hiding a weak property that matters more for the actual use.

Suppose two attribution methods achieve similar faithfulness scores. One is highly stable but coarse; the other is detailed but changes sharply under harmless input perturbations. For debugging, the detailed method may be preferable because it reveals localized failure modes. For a user-facing explanation, instability may be unacceptable because neighboring people receive incompatible narratives. Evaluation has no context-free winner because the objective includes the downstream role.

Organizations should also distinguish acceptance tests from research metrics. A paper may compare methods statistically across a benchmark. A production acceptance test should be phrased in terms of tolerances and failure actions. For example, a chosen explanation may be required to pass parameter-randomization checks, remain within a stability band across three approved backgrounds, and reproduce within a specified numerical tolerance under the same model version. If the test fails after

retraining, the explainer is not simply “a little worse”; the deployed explanation contract is broken.

The last component is review cadence. Explanations are not validated once. Data drift, model updates, library changes, and new user populations can alter their meaning. Periodic regression tests make explanation quality part of continuous assurance in the same way that accuracy, latency, and security are monitored over time.

There is no scalar XAI accuracy. Different metrics often disagree because they reward different properties and embed different assumptions. A mature evaluation therefore looks like a software test suite rather than a leaderboard: model-randomization tests, perturbation tests, reference sensitivity, subgroup analysis, adversarial stress, semantic ground truth where available, and user studies where human decisions matter.

Evaluation therefore becomes a portfolio of tests tied to an explanation contract, not a race for one metric. The next question is whether the validated artifact helps a person exercise better judgment rather than merely making the system sound more intelligible.

3.2 Human-centered explanation, oversight, and regulation

When explanations reach users, the problem changes from model analysis to interface design. A technically faithful artifact can still be incomprehensible, misleading, or mistimed. A clinician under time pressure does not need a twenty-feature importance ranking if the key question is whether one unstable variable drove an alert. A fraud investigator may need a global pattern first and the local details second. A citizen receiving an adverse decision may need a contrastive explanation and a route to contest it rather than an internal model diagnostic.

Progressive disclosure is therefore a powerful pattern. The first layer answers the operational question in plain language. A second layer exposes evidence, uncertainty, and alternative explanations. A third layer supports expert inspection with plots, provenance, and method details. This avoids the false choice between oversimplification and overwhelming transparency. It also makes it possible to give different audiences different views without pretending that every view is equivalent.

Explanations should be designed to calibrate reliance rather than maximize trust. Showing uncertainty, instability, or disagreement among methods can be more valuable than producing a single confident story. In some situations, the best explanation is a warning that the model is

outside its validated regime or that the evidence is underdetermined. Human-centered XAI is successful when it helps people know when to use the system, when to challenge it, and when to seek additional evidence.

Fairness also applies to the explanation interface itself. If explanation quality is systematically worse for minority groups, rare conditions, or atypical inputs, a nominally transparent system can reproduce inequity at the level of contestability. Evaluating subgroup explanation quality therefore belongs alongside subgroup predictive performance, especially when explanations affect access to appeals or human review.

A clinician, ML engineer, regulator, affected citizen, operations analyst, and autonomous software agent do not need the same explanation. Expertise changes vocabulary, acceptable complexity, and the kinds of evidence that support action. A radiologist may prefer concept-level evidence linked to image regions and comparable cases. An ML engineer may need gradients, ablations, training traces, and representation probes. A regulator may need documented constraints, model-level testing, and a reproducible audit trail.

Doshi-Velez and Kim proposed a useful hierarchy: application-grounded evaluation with real users and tasks, human-grounded evaluation with simplified human experiments, and functionally grounded evaluation using formal proxies. The hierarchy remains relevant because many XAI systems are evaluated only with proxy metrics even when their intended purpose is to change human decisions.

The purpose of an explanation is often said to be trust. This is poorly specified. The goal should not be maximum trust but calibrated reliance: users should know when the system is likely to be useful, when it is uncertain, when it may fail, and when human judgment should override it. An explanation that makes a weak model appear trustworthy is harmful even if users like it.

Good explanation interfaces help users build a mental model of system behavior. This can be measured by simulation tasks: after seeing explanations, can the user predict how the model will behave on new cases? Can the user identify a known failure mode? Can the user detect when a prediction relies on a shortcut? These are stronger tests than asking whether the interface feels understandable.

Natural-language explanations create a new design problem because fluency is itself persuasive. A generated rationale can fill causal gaps, suppress uncertainty, and present a coherent story even when the underlying computation is distributed, heuristic, or inconsistent. The

interface may therefore improve subjective understanding while reducing epistemic caution.

One practical defense is to separate evidence from narration. The system can present a natural-language summary, but the summary should be linked to structured evidence such as input attributions, retrieved sources, counterfactual tests, uncertainty estimates, or mechanistic probes. The prose should be treated as a view over evidence rather than as the evidence itself.

More detail is not always better. A full decision tree with thousands of nodes is transparent but not usable. A token-level attribution over a long document can overwhelm rather than clarify. Explanation systems need progressive disclosure: a compact summary, then drill-down into features, examples, counterfactuals, model behavior, and provenance as needed.

This suggests a layered interface architecture. The first layer answers the user's immediate question. The second provides supporting evidence. The third exposes uncertainty and alternative explanations. The fourth exposes technical diagnostics and reproducibility metadata. The deeper layers should remain accessible without forcing every user to read them.

Explanation systems can be uneven across groups even when the prediction model has similar aggregate performance. Local neighborhoods can be denser for majority populations, counterfactuals can be cheaper for some groups than others, and explanation vocabularies can align better with cultures represented in training data. Fairness therefore applies not only to predictions but to access to meaningful explanation and recourse.

A production evaluation should compare explanation stability, recourse cost, error discovery, and user comprehension across relevant subgroups. The goal is not to force identical explanations but to detect systematic differences that alter practical rights or decision quality.

High-stakes XAI lives inside governance rather than outside it. The EU AI Act illustrates the shift. Article 13 requires high-risk AI systems to be sufficiently transparent to enable deployers to interpret outputs and use them appropriately, while Article 14 requires effective human oversight supported by appropriate human-machine interface tools. These obligations do not mandate one universal XAI technique. They make the deployer's ability to understand and oversee the system part of the engineering problem.

That distinction matters. A heat map may be technically interesting yet irrelevant to oversight. An operator may need warnings about conditions of use, known limitations, confidence, data quality, drift, and the situations in which human intervention is required. A governance process may need

logs and model-version lineage more than feature attributions. Explainability therefore sits inside a larger transparency package that includes instructions, performance boundaries, monitoring, and accountability.

High-stakes explanations are also attractive attack surfaces. If a system reveals detailed feature dependence, an adversary may use that information to game the model. If explanations expose training examples, they can leak sensitive data. If a post-hoc explainer can be manipulated independently of the predictor, an organization may receive reassuring explanations while the underlying model behaves differently. Security analysis should therefore consider both the predictive channel and the explanatory channel.

The practical standard is proportionality. A low-risk recommender does not need the same evidential burden as clinical decision support. But whenever a person is expected to exercise meaningful oversight, the interface must give them information they can actually use. Human oversight that exists only in policy documents is not a technical control. Explainability becomes valuable when it changes the operator's ability to detect, question, and correct system behavior.

A realistic oversight workflow is layered. The operator first sees the decision, confidence, and a concise reason. They can then inspect the evidence, compare alternatives, and escalate to a specialist. The system records whether the operator accepted or overrode the output and why. This creates a feedback loop in which explanation quality can be evaluated from actual oversight behavior rather than assumed from interface design.

Regulatory transparency should therefore be connected to operational controls. Documentation that says a model is explainable is weak if the deployer cannot reconstruct a particular output or recognize when the system is outside its intended use. Conversely, a system can support strong oversight without revealing every parameter if it exposes validated decision evidence, limitations, uncertainty, and logs. The relevant question is functional: can the responsible human understand enough to exercise the responsibility assigned to them?

This functional view also avoids “compliance by explanation widget.” Adding a SHAP chart to a high-risk system does not by itself satisfy meaningful transparency, just as adding a log file does not automatically create human oversight. The interface, training, procedures, and technical evidence have to work together.

The EU AI Act makes transparency and human oversight operational requirements for high-risk systems. The regulation requires systems to be

sufficiently transparent for deployers to interpret outputs and use them appropriately, and it emphasizes understanding capabilities and limitations, guarding against automation bias, correctly interpreting outputs, and retaining the ability to override or stop the system. XAI can support these goals, but a feature-importance chart alone does not satisfy them.

The engineering implication is that explainability belongs inside an oversight design. Interfaces must expose enough information for the human to recognize failure modes, not merely enough to justify the model after the fact. Logging, uncertainty, contestability, role-specific training, and operational controls are part of explainability when the real objective is safe human oversight.

Explanations leak information. Repeated feature-attribution queries can reveal decision boundaries, sensitive training relationships, or internal model behavior. Counterfactuals can expose how to evade fraud or security controls. Example-based explanations can reveal private records. Mechanistic interfaces can expose safety circuits or exploitable weaknesses. Explainability therefore creates a genuine transparency-security trade-off.

Access controls should depend on the audience. Internal debugging tools can expose richer model information than public explanation APIs. Rate limits, query budgets, coarsened explanations, delayed disclosure, privacy-preserving aggregation, and differential privacy may be appropriate in sensitive settings. There is no principle that every explanation should reveal every available detail.

An explanation service should be tested as a system in its own right. Does it provide similar fidelity across demographic or operational groups? Are counterfactual costs systematically higher for some groups? Does a background dataset make minority cases look anomalous? Can explanations leak membership in the training set? These questions are separate from whether the predictor itself satisfies a fairness criterion.

The explainability pipeline can also become a fairness tool. Group-level attribution analysis, concept audits, and counterfactual comparisons can expose proxies and shortcuts. But explanation does not automatically remove bias; it only creates additional observability. Corrective action still requires changes to data, objective, policy, or model.

Human-centered design and regulation converge on a practical requirement: an explanation must support appropriate action and oversight. To satisfy that requirement after deployment, the entire explanation pipeline must be versioned, monitored, and defended as production infrastructure.

3.3 Production assurance: drift, security, privacy, and governance

The difference between research XAI and production XAI is reproducibility. In a notebook, an engineer can rerun an explainer interactively, adjust a baseline, and inspect the result. In production, an explanation may be reviewed months later by someone who was not present when the prediction was made. The model may have been retrained, the feature pipeline changed, the retrieval corpus updated, and the explainer library upgraded. Without lineage, the original explanation becomes impossible to reconstruct.

An explanation contract should therefore be materialized as data. Record the model identifier, input schema, preprocessing version, explainer and configuration, baseline or background set, target output, software version, timestamp, and relevant uncertainty measures. Where possible, retain references to the exact data or immutable hashes rather than only a rendered image. The visual artifact is a view; the explanation record is the evidence.

Explanation drift adds a monitoring dimension. Prediction distributions can remain stable while the model's internal reliance changes. A retrained fraud model might preserve aggregate performance but shift from transaction behavior toward device metadata. A medical model might maintain AUROC while becoming more sensitive to a site-specific acquisition artifact. Monitoring attribution distributions, concept activation, counterfactual costs, or other explanatory summaries can reveal such shifts. These signals should not be interpreted naively, because explainers themselves are unstable, but they can trigger deeper review.

The production pipeline therefore resembles an observability system. It samples explanations, stores evidence, compares cohorts and versions, runs regression tests, and routes anomalies to humans. The goal is not to generate explanations for every event at maximum detail. It is to make the system auditable at the right granularity and to preserve enough evidence to investigate when something important goes wrong.

A production explanation record should be immutable enough to survive organizational change. It should identify not only the model binary or checkpoint but also the feature schema, preprocessing graph, retrieval snapshot where relevant, explainer configuration, and the exact output target that was explained. This is especially important for multi-output systems and language models, where explaining one token, one sequence score, or one downstream decision are different tasks.

Regression testing can be built around a curated corpus of explanation cases. Include ordinary examples, known edge cases, previous incidents, subgroup-sensitive cases, and intentionally adversarial inputs. When the model or explainer changes, compare not only predictions but also explanatory artifacts and the results of their validation tests. Large unexplained shifts should trigger review even if aggregate performance is stable.

In production, an explanation is generated inside a changing system: models are retrained, features are transformed, services are deployed in multiple regions, data distributions drift, and users see different interface layers. Reproducibility therefore requires more than saving a PNG. The explanation must be associated with the exact model artifact, preprocessing pipeline, output target, explainer configuration, background dataset, relevant data version, and software versions.

This suggests treating explanations as structured artifacts. A local explanation can be stored as an object containing the prediction, explanatory method, semantics, parameters, evidence, uncertainty, and links to validation results. Such an object can be rendered differently for an engineer, regulator, or end user without losing provenance.

A useful engineering abstraction is an explanation contract. The contract states the question being answered, the domain in which the explanation is valid, the model and data dependencies, the assumptions of the method, the stability expectations, the privacy policy, and the validation tests that the artifact must pass. The contract turns vague requests such as 'make the model explainable' into testable system requirements.

ExplanationContract:

```
question = "Which inputs locally control this decision?"
target = "approved_probability"
method_family = "feature_attribution"
reference_policy = "validated_background_v4"
valid_model_versions = ["credit-v18"]
required_tests = ["model_randomization", "baseline_sensitivity",
"local_ablation"]
max_explanation_drift = "defined by cohort-specific tolerance"
privacy_class = "internal-sensitive"
human_view = "top concepts + counterfactual context"
```

The syntax above is deliberately simple. The important idea is that explanation semantics and tests become machine-readable. A CI pipeline can reject a deployment when an explainer no longer satisfies its contract, just as it rejects a failing unit test.

Traditional model monitoring tracks input drift, output drift, accuracy, calibration, latency, and sometimes fairness. Explanation monitoring adds another signal: changes in the model's reliance on features, concepts, examples, or internal mechanisms. A classifier can preserve top-line accuracy while shifting from a robust signal to a spurious proxy. Attribution drift can reveal that change before conventional metrics do.

Explanation drift must be interpreted carefully because explainer variance can look like model drift. The monitoring baseline should therefore be established on fixed reference sets, with repeated explanations where stochasticity matters. The system should distinguish changes caused by data, model parameters, preprocessing, and explainer configuration.

A robust production workflow begins with behavioral model tests, then generates explanations, runs sanity and randomization checks, evaluates fidelity and stability, compares relevant subgroups, tests adversarial manipulation, validates causal claims where recourse is provided, and finally performs application-grounded human evaluation when explanations influence people. This ordering matters: it is wasteful to study whether humans like an explanation before establishing that the artifact contains meaningful information about the model.

Deploying explanations creates new risks. Detailed local explanations can help attackers discover decision boundaries. Training-data attribution can expose sensitive records. Counterfactuals can reveal how to game a fraud or eligibility system. Generated rationales can create a false sense of transparency. Even explanation stability can become a target if an attacker learns which perturbations leave the displayed story unchanged.

This means security and privacy should influence explainer selection. Public-facing explanations may need coarser granularity than internal debugging tools. Sensitive training examples may need aggregation or access control. Explanations used for incident response may be retained under stricter governance than ordinary telemetry. Where explanations are generated by another model, the generated text should be linked to machine-checkable evidence rather than treated as authoritative by itself.

The tooling ecosystem is mature enough to support serious workflows but not mature enough to eliminate methodological decisions. SHAP remains widely used for Shapley-style attribution. Captum provides PyTorch-native attribution and layer-analysis methods. InterpretML

includes Explainable Boosting Machines and post-hoc explainers. Quantus offers a growing metric suite for evaluation. OpenXAI provides benchmark-oriented comparison. The right tool is the one whose semantics and failure modes fit the contract; a popular library name is not a validation argument.

For organizations, the most important tooling decision may be architectural rather than algorithmic: build a common evidence interface across models. If every team logs explanations differently, cross-model governance becomes impossible. A shared schema for model identity, inputs, explanation semantics, uncertainty, provenance, and test results makes explainability comparable and reviewable even as individual methods evolve.

Tooling should be versioned as carefully as models. Explanation libraries change default behaviors, supported maskers, numerical approximations, and evaluation metrics. A result produced by “SHAP” or “Captum” is not reproducible unless the relevant explainer class, background data, framework versions, and options are captured. In regulated or high-stakes settings, software-bill-of-materials practices should extend to the explanation stack.

There is also a build-versus-buy decision. Cloud platforms increasingly expose generic explanation features, which can reduce integration effort. They do not remove the need to validate semantics against the application. A managed service may calculate feature importance correctly while the chosen background or data encoding makes the explanation unsuitable for the domain. Convenience should not be mistaken for epistemic validation.

A small centralized XAI/assurance layer can therefore be more valuable than many disconnected team-specific notebooks. It can provide approved configurations, common logging, evaluation templates, and access controls while still allowing specialized methods where justified. The objective is consistency of evidence, not uniformity of algorithms.

Explanations leak information. Repeated feature-attribution queries can reveal decision boundaries, sensitive training relationships, or internal model behavior. Counterfactuals can expose how to evade fraud or security controls. Example-based explanations can reveal private records. Mechanistic interfaces can expose safety circuits or exploitable weaknesses. Explainability therefore creates a genuine transparency-security trade-off.

Access controls should depend on the audience. Internal debugging tools can expose richer model information than public explanation APIs. Rate limits, query budgets, coarsened explanations, delayed disclosure,

privacy-preserving aggregation, and differential privacy may be appropriate in sensitive settings. There is no principle that every explanation should reveal every available detail.

An explanation service should be tested as a system in its own right. Does it provide similar fidelity across demographic or operational groups? Are counterfactual costs systematically higher for some groups? Does a background dataset make minority cases look anomalous? Can explanations leak membership in the training set? These questions are separate from whether the predictor itself satisfies a fairness criterion.

The explainability pipeline can also become a fairness tool. Group-level attribution analysis, concept audits, and counterfactual comparisons can expose proxies and shortcuts. But explanation does not automatically remove bias; it only creates additional observability. Corrective action still requires changes to data, objective, policy, or model.

The open-source ecosystem now covers most classical explanation families. SHAP remains the dominant general feature-attribution framework, especially for tree and tabular models. Captum provides PyTorch-native attribution and layer analysis, including transformer and LLM workflows. InterpretML supports glass-box models such as Explainable Boosting Machines as well as black-box explainers. Quantus collects many evaluation metrics, while OpenXAI provides standardized benchmarking and leaderboards.

Managed platforms from major cloud vendors integrate feature attribution, error analysis, fairness, counterfactuals, and governance to varying degrees. These interfaces reduce operational friction but do not validate the semantics of the underlying method. A polished cloud dashboard inherits the same baseline, dependence, and causal assumptions as the algorithm it wraps.

Once explanation becomes a production service, provenance and continuous testing are unavoidable. This infrastructure also prepares us for foundation models, where the most persuasive explanatory artifact—the model’s own language—can be the easiest one to overinterpret.

Chapter 4. Explaining Foundation Models

Foundation models force XAI to confront an uncomfortable fact: fluent language makes weak evidence look strong. A model can produce a coherent account of why it answered as it did without that account being a faithful record of the computation that produced the answer. At the same time, modern models expose internal representations rich enough to support probes, interventions, sparse feature dictionaries, causal tracing, and increasingly sophisticated mechanistic hypotheses. This chapter builds an evidence hierarchy for those claims. It distinguishes generated rationale from behavioral evidence, representation-level evidence from causal intervention, and useful mechanistic models from literal descriptions of every computational detail. The goal is not to make large models simple. It is to make claims about them proportionate to the evidence we actually possess.

QUESTION TO CARRY THROUGH THE CHAPTER — Which parts of the explanation are observable outputs, which are behavioral evidence, and which are causal claims about internal computation?

4.1 Generated rationales are not privileged access to reasoning

Language models can describe their own reasoning with extraordinary fluency. That ability is useful, but it tempts users to assume that the generated narrative is a transparent readout of the internal computation. Research on chain-of-thought faithfulness repeatedly warns against that assumption. Models can omit influential hints, rationalize answers after the fact, or produce reasoning text that changes under prompting while the answer remains stable. Recent work probing reasoning models continues to find discrepancies between reported and behaviorally inferred influences.

The correct interpretation is modest: chain-of-thought is an output behavior that may contain useful intermediate structure. It can improve task performance, expose obvious errors, and support process supervision. It can sometimes provide evidence about how a model is solving a problem. It is not privileged introspection. Treating it as such

collapses the distinction between a model producing a plausible explanation and an investigator establishing a causal account.

For engineers, this suggests simple tests. Change an apparently irrelevant hint and see whether the answer changes without the rationale acknowledging it. Remove a cited source and test whether the conclusion persists. Paraphrase the prompt, alter the order of evidence, or insert distractors. Compare hidden-state or activation interventions with the claims in the generated rationale. If the textual reasoning says one thing while controlled behavior says another, the behavior deserves more evidential weight.

LLM explainability therefore begins outside the model. We can measure prompt sensitivity, token influence, source dependence, retrieval effects, and response consistency before attempting to infer internal mechanisms. These tests are comparatively cheap, scalable, and directly tied to system behavior. They form the lower rungs of an evidence ladder that can later be extended with probes and mechanistic experiments.

The distinction becomes even more important for models trained specifically to emit long reasoning traces. A model can learn that certain styles of reasoning are rewarded, just as it learns any other output convention. The trace may correlate strongly with useful internal computation without being a literal transcript of it. The stronger the training pressure on the visible rationale, the more carefully faithfulness should be tested rather than assumed.

One practical audit is “reasoning perturbation without answer perturbation.” If the visible chain can be radically changed by stylistic prompts while the answer and internal behavioral sensitivities remain stable, the trace is partly presentation. Another is “hidden influence.” Insert information that changes the answer and test whether the rationale acknowledges it. The combination of such tests gives a more grounded picture than judging the plausibility of the prose.

For many applications, the safest interface is to ask the model for concise justifications linked to external evidence rather than unrestricted claims about internal reasoning. The explanation then focuses on what can be checked: source support, assumptions, intermediate calculations, tool outputs, and uncertainty.

Classical XAI often assumes a fixed prediction for a fixed-dimensional input. An LLM is an autoregressive conditional generator with distributed representations, long contexts, tool calls, retrieval, system prompts, sampling, and potentially hidden state in an agent harness. Its behavior can depend on tokenization, context order, decoding parameters, external

tools, and prior interactions. The explanatory target may be a token, a sentence, a plan, a refusal, a factual claim, or a multi-step action.

Chain-of-thought deserves special care because the output is linguistically similar to the kind of explanation a person would give. That similarity is cognitively powerful but epistemically weak. A generated rationale can be conditioned by prompting conventions, can omit features that affected the answer, can incorporate features that did not, and can be post-hoc in the ordinary sense that the model produces a plausible narrative after the relevant decision has effectively been determined. Intervention studies showing unfaithful rationales do not imply that chain-of-thought is useless. They imply that it should be interpreted as behavior that can itself be studied, not as a privileged trace of hidden computation.

This distinction matters operationally. A rationale can still improve usability, expose assumptions, help a user notice an obvious inconsistency, or provide a compact interface to retrieved evidence. Those are legitimate roles. The error is to merge them with the mechanistic claim that the displayed sequence is the causal path used by the model. An engineering system should therefore label generated reasoning as reported reasoning or generated rationale unless independent tests support a stronger statement.

A second complication is that a foundation model's behavior depends on more than its weights. Prompt construction, system instructions, retrieval results, tool schemas, conversation state, sampling parameters, and surrounding code all shape the output. Consequently, an LLM explanation that ignores the runtime context can be accurate about the neural model and still wrong about the deployed system. This is one reason provenance becomes central as soon as models are embedded in applications.

At the same time, the model can produce a compelling natural-language account of why it answered as it did. This tempts users to treat verbalized reasoning as direct introspection. Research has shown that this inference is unsafe.

Turpin and colleagues demonstrated that chain-of-thought explanations can systematically misrepresent factors that influenced a model's answer. Later intervention studies on reasoning models found that hints or biasing cues could affect answers without being reliably acknowledged in the generated reasoning. The appropriate conclusion is not that chain-of-thought has no value. It can improve task performance, support debugging, and communicate a useful decomposition. But it should be

treated as generated behavioral evidence rather than a transparent dump of hidden computation.

This distinction becomes even more important when models are optimized for helpfulness or evaluated on the quality of their rationales. A model can learn to produce explanations that satisfy users or evaluators independently of the mechanism that caused the answer. Plausibility can become a trained capability.

Classical attribution can be adapted to LLMs. Engineers can measure how prompt tokens influence the probability of a target continuation, use Integrated Gradients over embeddings, mask context spans, or compare completions after counterfactual edits. Captum and other libraries provide practical implementations. These methods can answer useful questions such as which context passages influence a generated answer or which prompt sections trigger a refusal.

The challenges are substantial. Language is discrete, tokenization fragments semantic units, context features interact strongly, and removing a token often creates an unnatural prompt. Autoregressive generation compounds effects across time: a small influence on an early token can redirect the entire continuation. Attribution to one final sequence therefore depends on the decoding path and target definition.

Generated rationales are therefore one evidence layer among several. A stronger account of foundation-model behavior combines behavioral counterfactuals, source interventions, representation analysis, and, where feasible, causal tests of internal computation.

4.2 A layered evidence model for prompts, sources, representations, and mechanisms

A useful response to the unreliability of generated rationales is not to abandon explanation, but to separate evidence into layers with different semantics.

Foundation-model explanation works best as a layered investigation rather than a single method. At the first layer are output artifacts: rationales, citations, self-reported confidence, and structured intermediate products. They are easy to obtain and useful for interaction, but weak as evidence of mechanism. The next layer is behavioral: prompt perturbations, context ablations, source removal, counterfactual instructions, and repeated sampling. These tests ask what the system actually depends on.

Representation probes form another layer. A linear probe may reveal that information about syntax, entities, truthfulness, or some domain

concept is decodable from internal states. Decodability is not the same as use. Causal interventions such as activation patching, representation editing, or ablation ask whether manipulating an internal state changes behavior. Mechanistic work goes further by proposing circuits or feature interactions that can be tested across examples. Formal verification, where applicable, provides an even stronger but much narrower form of evidence.

Retrieval-augmented systems add provenance. A citation can show which document supports an answer, but source attribution is not reasoning attribution. A model may cite a document it did not materially use, use a retrieved source without citing it, or synthesize an answer through information distributed across several documents. A robust audit therefore records retrieval candidates, selected context, prompt construction, output, citations, and ablation results. The question becomes: which sources were present, which were necessary, and which claims are actually entailed by them?

This layered view prevents two opposite errors. One is dismissing all model-generated reasoning as useless because it is not guaranteed faithful. The other is treating fluent rationales as complete explanations. Output reasoning is valuable when it is explicitly positioned as one piece of evidence and connected to stronger tests when the stakes justify them.

A layered evidence model also helps teams allocate cost. Behavioral ablations can be run routinely on a sample of production cases. Probes and activation interventions may be reserved for incident analysis or model research. Full mechanistic studies may target a narrow capability or known failure mode. Formal analysis may apply only to a small safety-critical component. There is no requirement that every prediction climb to the top of the ladder.

This table prevents a common escalation in language: moving from “the model said” to “the model represented” to “the model used” without adding evidence. Each row requires a stronger experimental basis.

Table 4.1 — Evidence about an LLM should be labeled by the strength of the claim it can support.

Concept / object	Engineering meaning
Generated rationale	Observable model output that may be useful to users but is not privileged access to hidden computation.

Concept / object	Engineering meaning
Behavioral test	A prompt, source, or context intervention showing that a behavior changes when a relevant external condition changes.
Representation probe	Evidence that internal states contain information correlated with a concept or property.
Causal intervention	Evidence that changing an internal representation changes the behavior of interest under controlled conditions.
Mechanistic or verified claim	A structured hypothesis or bounded property supported by interventions, replication, or formal verification.

The hierarchy is not a ranking of usefulness. A source ablation can be more operationally valuable than a mechanistic circuit for many applications. The point is to keep the vocabulary aligned with what the experiment actually established.

The architecture should preserve links between layers. A generated statement such as “the answer depends mainly on document A” can be accompanied by retrieval metadata and a source-ablation score. A mechanistic claim can point to the prompts on which the circuit was tested. This creates a trace from readable explanation to stronger evidence without forcing every user to inspect the lower-level artifacts.

Figure 9.1. An evidence ladder for foundation-model explanation. Higher layers are stronger but narrower and more expensive.

A probe trains or defines a simple readout from hidden activations to a property of interest. Probes can reveal whether information about syntax, entities, sentiment, truthfulness, location, or another concept is linearly accessible at a layer. This is useful for mapping where information becomes available during processing.

The lower levels are cheap and often useful, but they support weaker claims. Moving upward requires stronger intervention and more careful replication. The ladder is therefore a vocabulary discipline as much as a methodological one.

Evidence ladder for foundation models

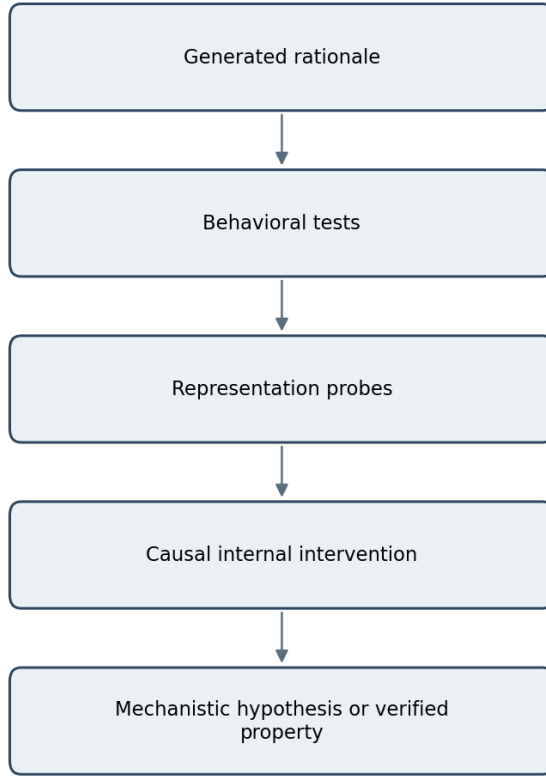


Figure 4.1 — The evidence ladder keeps foundation-model explanations from claiming more than their experiments establish.

The purpose is not to insist that every application reach the top. Many production questions are answered adequately by behavioral tests and provenance. Mechanistic and verified claims are appropriate when the consequence, research objective, or safety requirement justifies the additional cost.

Availability is not use. A representation can contain information that the output mechanism ignores. A sufficiently expressive probe can also recover weak information that is not functionally meaningful. Probe complexity, control tasks, layer comparisons, and causal interventions are therefore needed to move from 'the information is decodable' toward 'the information participates in computation.'

Behavioral counterfactuals are often the most practical LLM explanation tool because they require no weight access. Change an instruction, remove a retrieved passage, swap entity names, alter ordering, paraphrase the query, or inject a controlled distractor, then observe whether the answer changes. Carefully designed perturbation suites can reveal instruction hierarchy, reliance on retrieved evidence, positional bias, sycophancy, shortcut use, and brittleness.

The principle is experimental: change one factor while preserving the rest of the relevant semantics. This is difficult in natural language because every rewrite can introduce new cues. Multiple paraphrases and generative controls reduce the risk of attributing an effect to wording rather than meaning.

Retrieval-augmented generation introduces a distinct and often more tractable provenance problem. A system can record which documents were retrieved, which passages were supplied to the model, and which claims are supported by those sources. This improves traceability even when the internal language-model computation remains opaque.

However, citing a source does not prove that the source caused the answer, nor that the model interpreted it correctly. A robust system can test source necessity by removing the passage, source sufficiency by retaining only selected evidence, and claim entailment by independent verification. Retrieval provenance and mechanistic explanation should be kept conceptually separate.

The evidence ladder in Figure 10.1 is not a total ranking of usefulness. Output rationales can be the best interface for a user even though they are weak causal evidence. Behavioral attribution is stronger for questions about input dependence. Representation probes expose internal information. Activation patching and ablation provide causal evidence about internal computation. Circuit hypotheses connect multiple components into a mechanism. Formal verification can establish a narrow property with the strongest guarantee, but usually for much smaller scope.

A mature LLM assurance system combines layers. It may expose a human-readable rationale, link claims to sources, run behavioral counterfactuals, inspect selected internal features, and require verified policy constraints for high-risk actions. No single technique has to carry the entire burden of explanation.

The evidence ladder does not guarantee a unique mechanism, but it prevents a weak observation from being described with the vocabulary of a stronger one. Mechanistic interpretability occupies the upper middle of that ladder: it proposes internal computational structures and asks whether interventions support them.

4.3 Mechanistic interpretability: circuits, interventions, and sparse features

Mechanistic interpretability becomes intuitive if we imagine debugging a compiled program without source code. We can inspect memory, trace execution, patch variables, and infer modules from behavior. Neural networks are more difficult because their internal variables are distributed, learned, and often polysemantic. Still, the experimental logic is similar: observe internal states, hypothesize a computation, intervene, and test whether the predicted behavior changes.

Activation patching is a particularly useful example. Run the model on a clean input and a corrupted input. Replace an activation from one run with the corresponding activation from the other. If the correct behavior is restored, the patched component carries causally relevant information under that intervention. Causal tracing generalizes this style of reasoning. Patchscopes provide a broader framework for inspecting hidden representations by transforming and reusing them through the model. These techniques move mechanistic analysis beyond correlation, but they remain local experiments rather than complete proofs of a circuit.

Superposition complicates interpretation because a network may encode more features than there are available dimensions by allowing features to share representational space. Individual neurons are therefore often poor semantic units. Sparse autoencoders attempt to recover a larger dictionary of features whose activations are sparse and more interpretable. The 2025 survey literature reflects how quickly this area has expanded, including questions about training objectives, feature quality, cross-layer structure, steering, and evaluation.

The central caution is that an SAE feature is itself the output of another learned model. A feature that appears monosemantic on sampled activations may behave differently elsewhere. Labels generated by language models can be convenient but circular if they are accepted without intervention. Mechanistic interpretability becomes scientifically strong when semantic labels, causal effects, generalization across contexts, and failure cases are evaluated together.

Early interpretability work visualized neurons, attention heads, activation patterns, and feature maps. Mechanistic interpretability adds a stronger ambition: identify functional components, infer how they interact, and test the proposed mechanism through interventions. The analogy is reverse engineering a compiled program whose source code was produced by optimization rather than written by a human.

This framing is especially important for foundation models because broad behavioral tests cannot enumerate every possible context. If a safety-relevant mechanism can be localized and understood, engineers may be able to test it across distributions, monitor it, or modify it more directly. The challenge is scale: a frontier model contains billions of parameters and vast numbers of possible activation patterns.

Circuit analysis seeks small computational subgraphs that implement recognizable behavior. A classic line of work identified transformer attention patterns such as induction heads. Causal tracing and model-editing work such as ROME introduced intervention techniques for locating where factual associations appear to be mediated. The core idea is to corrupt or alter representations, restore selected internal states, and observe whether the target behavior returns.

Causal intervention is stronger evidence than correlation. If restoring a hidden state restores a fact, that component is implicated in the computation. Yet localization remains subtle. Later studies found that locations identified by causal tracing do not always correspond cleanly to where an edit is most effective. Mechanisms can be distributed, redundant, or context-dependent.

Activation patching replaces hidden activations from one run with activations from another and measures the effect. By constructing clean and corrupted prompts, researchers can test which layers, positions, heads, or features carry information required for a behavior. Patchscopes generalizes the idea by transplanting hidden representations into carefully designed target contexts that make their information legible through the language model itself.

Mechanistic interpretability should be understood as model building about a model. Researchers observe activations, learn dictionaries, identify circuits, propose semantic labels, and test whether interventions behave as the hypothesis predicts. The resulting mechanism is not automatically the one true decomposition of the network. Different abstractions can be simultaneously useful at different scales, just as a program can be described at the level of machine instructions, functions, modules, or algorithms. The engineering question is whether the chosen abstraction supports reliable predictions and interventions over the scope for which it is claimed.

Sparse autoencoders make this issue vivid. They can replace dense activation patterns with a larger set of sparse features that often admit clearer semantic descriptions. This is a genuine advance because it changes the search space for hypotheses. Yet feature sparsity does not prove semantic purity, and a human-readable label can hide polysemy,

context dependence, or distributed interactions. The proper validation loop is therefore semantic discovery followed by controlled activation tests, counterexamples, cross-context checks, and comparison with alternative decompositions.

SemanticLens and related approaches add another layer: they map components into semantic spaces derived from foundation models so that large networks can be searched and compared at scale. The gain is operational. Instead of manually inspecting thousands of neurons or channels, a team can ask where a concept appears, whether a suspicious concept recurs across checkpoints, or which components differ between model versions. The risk is also operational: semantic search can create a feeling of understanding before causality has been established. Search discovers candidate mechanisms; intervention remains the stronger test.

Circuit tracing and attribution graphs attempt to recover interacting internal features that contribute to a behavior. Their promise is that an explanation can move beyond single features toward a structured computation. Their limitation is that the graph is itself produced by approximations and thresholds. A graph that looks algorithmic should not be read as source code extracted from the network. Its value lies in whether it predicts the effect of targeted interventions, generalizes across relevant cases, and helps discover failure modes that simpler analyses miss.

These methods create an experimental laboratory inside the network. They are powerful because the intervention occurs on the actual computation. They are expensive because the search space is enormous and because every experimental design encodes assumptions about what is being isolated.

A major obstacle is polysemanticity: one neuron can respond to multiple unrelated patterns, and one semantic feature can be distributed across many neurons. This is often described through superposition, where a representation space stores more features than there are directly interpretable dimensions by using overlapping directions.

Sparse autoencoders attempt to decompose dense activations into a larger set of sparse latent features. Instead of treating neurons as the atomic units of meaning, the model learns a dictionary of directions that reconstruct activation patterns while activating sparsely. Work from Anthropic and independent researchers has found many features with recognizable semantic behavior, making sparse dictionary learning one of the most active approaches in LLM interpretability.

The attraction of sparse autoencoders can obscure an important fact: the sparse feature dictionary is itself a learned model. It introduces

reconstruction error, depends on training data and sparsity choices, and may produce different decompositions for different runs. A feature that looks monosemantic under inspected examples may have unobserved activations or split into multiple refinements under a larger dictionary.

Mechanistic claims built on sparse features therefore need validation at two levels. First, the learned feature must faithfully represent an aspect of the original activation space. Second, interventions on the feature must cause the predicted changes in model behavior. Feature visualization alone is insufficient.

Mechanistic explanations become valuable when they predict the effect of internal change. Scaling that process requires semantic search, automated hypothesis generation, and careful record keeping—but scale must not be confused with certainty.

4.4 Scaling mechanistic understanding without pretending the map is the territory

The central scaling challenge is not merely to extract more internal features. It is to preserve the distinction between semantic search, causal evidence, and a complete account of computation.

Scaling mechanistic analysis requires more automation than a researcher manually naming neurons. SemanticLens is an example of this direction: it maps network components into a semantic foundation-model space so that components can be searched, compared, and labeled at scale. The goal is not only explanation in the conversational sense but validation: detecting spurious concepts, comparing representations across models, and making component-level analysis tractable for large systems.

Attribution graphs and circuit-tracing tools take another route. They attempt to reconstruct paths of influence through learned features for a particular output. Anthropic's 2025 open-source circuit-tracing work made this style of analysis more accessible for open-weight models and helped popularize the idea of an attribution graph as a partial internal causal story. The word partial matters. Large models can contain redundant pathways, distributed computations, and context-dependent mechanisms. A graph can be highly informative without being a complete execution trace.

Natural-language autoencoder work pushes the abstraction further by learning human-readable descriptions of activation patterns. This is attractive because the bottleneck becomes semantic rather than merely sparse. But it raises a familiar XAI question at a new level: are the descriptions faithful to the representation, or merely plausible summaries?

The same experimental discipline applies. Intervene on the represented feature, predict behavior, test out-of-distribution examples, and compare alternative descriptions.

The frontier is therefore moving from visualization toward mechanistic validation. The strongest future systems may combine automatically discovered features, semantic labels, causal interventions, and formal or statistical tests into a continuously maintained model of mechanisms. Such a model would not “explain the whole network” in one shot. It would accumulate validated local theories of how specific capabilities and failure modes are implemented.

Semantic automation also creates a new risk: explanations can become model-on-model agreements. If one foundation model labels the internal features of another, and a second language model judges those labels, the pipeline may look sophisticated while sharing blind spots. Independent evidence is therefore essential. Human expert review, synthetic concept tests, controlled interventions, cross-model comparison, and prediction of held-out behavior all provide ways to escape purely linguistic self-consistency.

A practical mechanistic study should have a stopping rule. It is easy to keep discovering ever more detailed components without knowing when the explanation is good enough. One useful criterion is predictive sufficiency: can the proposed mechanism predict what the model will do on new carefully chosen inputs and predict the effect of interventions? Another is compression: does the mechanism explain a meaningful class of behavior with fewer assumptions than a list of isolated observations? These criteria move the work toward science rather than taxonomy.

The eventual engineering payoff may be mechanism-level monitoring. If a validated feature or circuit is associated with a dangerous shortcut, a production system could monitor its activation or periodically retest the causal relationship after model updates. This is still frontier research, but it illustrates why mechanistic interpretability matters beyond intellectual curiosity.

Figure 10.1. Scalable mechanistic interpretation connects feature discovery to semantics and causal validation.

A complementary 2025 direction is SemanticLens, introduced by Dreyer and colleagues. It maps neural components into the semantically structured multimodal space of a foundation model, enabling textual search over components, automated labeling, cross-model comparison, and audits against domain expectations. The work demonstrates the possibility of moving from manual neuron browsing toward scalable component-level validation.

Scaling mechanistic work creates a problem of scientific logistics as much as one of algorithms. A large model can expose millions of candidate features, billions of activation events, and an effectively unbounded set of prompts. Human inspection cannot be the primary validation mechanism. The scalable architecture therefore needs automated hypothesis generation, sampling strategies that search for counterexamples, reproducible intervention harnesses, and databases that retain which model version, layer, feature dictionary, prompt set, and metric produced each claim.

This is where mechanistic interpretability begins to resemble software testing. A feature labeled 'refusal behavior' should come with positive examples, negative examples, contexts where the label fails, causal interventions, and regression tests across model updates. A circuit hypothesis should include not only a diagram but also executable perturbations that reproduce the claimed effect. The artifact becomes stronger when another team can rerun the intervention and observe the same bounded behavior.

Automation will not remove the need for judgment. Semantic labels can be generated by language models, counterexample prompts can be generated by agents, and candidate circuits can be ranked automatically, but those systems inherit their own failure modes. The practical goal is not an autonomous oracle that explains a frontier model. It is a research instrument that can generate, challenge, and version hypotheses faster than manual analysis alone while preserving enough provenance for experts to audit the process.

There is also a strategic reason to resist premature mechanistic certainty. Foundation models are changing architecture, training regime, modality, context length, and tool use faster than interpretability methods stabilize. A vocabulary that works for one model family may not transfer cleanly to another. The durable engineering practice is therefore to preserve the evidence hierarchy: behavioral evidence at one level, representation evidence at another, causal intervention above it, and formal guarantees only where the property and model permit them.

SemanticLens also illustrates a recurring frontier pattern: use one powerful representation model to interpret another. This can scale semantic analysis, but the interpreter model imports its own biases and blind spots. The semantics of the explanation are therefore mediated by another learned system. Cross-checking with direct interventions and domain ground truth remains necessary.

Anthropic's 2025 circuit-tracing work constructed attribution graphs intended to reveal computational pathways through sparse features. Case

studies explored factual recall, multilingual behavior, planning, arithmetic, hallucination-related mechanisms, refusal behavior, and aspects of chain-of-thought faithfulness. The scientific value lies in moving from isolated features toward connected causal hypotheses.

These results should not be mistaken for complete reverse engineering. Attribution graphs select and approximate parts of a computation; feature dictionaries are imperfect; causal interactions can be missed; and validating a proposed circuit across contexts remains difficult. Mechanistic interpretability is best treated as an experimental science whose hypotheses are increasingly powerful but still local and incomplete.

In 2026, Anthropic introduced Natural Language Autoencoders, which place a natural-language bottleneck between an activation verbalizer and a reconstructor. The method aims to produce unsupervised natural-language descriptions of activations while preserving enough information to reconstruct them. Conceptually, this is a striking step: the latent explanatory units are no longer only vectors with human-added labels but textual descriptions generated as part of the representation system.

The opportunity is a scalable semantic interface to hidden states. The risk is another layer of learned translation. A verbalizer can omit information, overgeneralize, or produce descriptions optimized for reconstruction rather than scientific truth. Natural-language feature descriptions therefore require the same separation between plausibility and faithfulness that applies to model rationales.

The responsible position is neither that mechanistic interpretability has solved deep models nor that it is merely decorative. It is becoming an experimental science of internal computation. Chapter 5 extends the same evidential discipline beyond the model boundary to multimodal and agentic systems.

Chapter 5. From Models to Systems

Modern AI systems are increasingly larger than a single model invocation. They combine text, images, audio, retrieval, tools, memories, policies, other agents, and long interaction histories. In such systems the phrase 'explain the model' is too narrow. The operational question becomes 'reconstruct the episode and identify which dependencies were necessary for the outcome.' Multimodal XAI introduces cross-representation alignment problems; agents introduce temporal and environmental causality; formal verification introduces the possibility of certifying narrow explanatory properties. This chapter treats these as one transition: from explaining a mathematical function to explaining a system that acts in the world.

QUESTION TO CARRY THROUGH THE CHAPTER — What evidence is required to reconstruct a system-level episode across modalities, tools, agents, and time?

5.1 Multimodal explanations and cross-modal evidence

Multimodal explanation is not just XAI repeated once per modality. When image, text, audio, video, or structured signals are fused, the interesting question is often how evidence moves between them. Did the model answer because of a visual region, a caption, or the agreement between the two? Did text dominate because the image was ambiguous, or because the architecture learned a shortcut? A separate heat map for each modality can miss the interaction that actually matters.

The 2026 systematic survey by Giovannoni and colleagues illustrates both the growth and immaturity of the field. Their review covers 155 publications and 54 datasets from 2018–2025, with healthcare representing 48 percent of the surveyed application literature. They also report limited public code and limited human evaluation, a sign that multimodal XAI is expanding faster than its validation practices. The lesson for engineers is to resist importing single-modality metrics without testing whether they capture cross-modal dependence.

Useful experiments include modality ablation, swapping one modality while keeping another fixed, counterfactual recombination, and measuring whether cross-attention or fusion representations causally affect the output. In a medical system combining an image and a clinical note, for example, one can remove the note, replace it with a contradictory note, or preserve the note while modifying a localized image finding. The explanation becomes a matrix of dependencies rather than one attribution vector.

Generated multimodal rationales can improve usability by turning evidence into text or visual overlays, but they add another model layer that can hallucinate coherence. The safest architecture links the generated explanation to inspectable evidence: highlighted regions, source snippets, modality ablations, uncertainty, and provenance. The narrative becomes a presentation layer over tests rather than the test itself.

A multimodal safety case should explicitly test conflicts between modalities. If an image suggests one diagnosis while a note suggests another, does the model behave predictably? If audio and transcript disagree, which channel dominates and why? If metadata is removed, does the model suddenly rely on a visual shortcut? These conflict cases are often more informative than ordinary examples because they reveal the fusion policy learned by the system.

The explanatory unit can also change over time. In video, an isolated frame may be irrelevant while a temporal pattern is decisive. In sensor fusion, two individually weak signals may become informative only when synchronized. A good explainer must therefore represent interactions across modality and time rather than forcing everything into independent importance scores.

Evaluation should include modality-complete and modality-degraded conditions. Real systems lose sensors, receive corrupted text, encounter missing metadata, or operate with asynchronous streams. Explanation quality under these conditions is part of robustness, especially if operators depend on the explanation to decide whether the model can still be trusted.

Vision-language models can answer a question correctly while relying mostly on text priors, mostly on image cues, or on a genuine interaction between the modalities. A heat map over the image plus a verbal rationale does not reveal which case occurred. Multimodal explainability must therefore measure both within-modality importance and cross-modal dependence.

MM-SHAP is one example of treating modality use as an explanatory target: Shapley-style analysis estimates how much the visual and linguistic inputs contribute. Other work uses image masking, text counterfactuals, cross-modal representation probes, and synthetic benchmarks. The general test is whether changing one modality alters behavior in a way predicted by the explanation while the semantic content of the other is controlled.

Vision-language systems can generate natural-language justifications that mention objects, relations, and visual evidence. Datasets such as e-ViL helped establish this task. As with LLM rationales, textual plausibility is not causal evidence. A model can describe an object correctly because its language prior makes the description likely even if the relevant visual pathway did not drive the answer.

Future multimodal XAI therefore needs paired evidence: textual claims linked to localized visual support, necessity tests that remove the cited region, sufficiency tests that preserve only the claimed evidence, and internal cross-modal interventions that probe how image and language representations interact.

Multimodal explanation therefore requires a theory of correspondence: which evidence in one modality supports a claim about another, and how that correspondence is tested. Agents add a further dimension because explanations must now reconstruct actions through time.

5.2 Agentic systems: provenance, replay, and causal intervention

Agents change the unit of explanation from a prediction to an episode. A coding agent may inspect files, search documentation, edit code, run tests, revise its plan, and open a pull request. A research agent may retrieve papers, create intermediate notes, run analyses, and synthesize a report. Explaining the final output requires reconstructing information flow and action selection across time.

A raw log is necessary but insufficient. Traceability tells us what happened: which prompt was sent, which tool was called, what result returned, what state changed. Explanation asks why a particular action became preferred and what alternatives would have changed the trajectory. The two can be combined through replay. Freeze the environment, remove one retrieved document, alter one tool result, replace one intermediate state, and see whether the agent still reaches the same conclusion. This is the system-level analogue of perturbation and activation patching.

Long-horizon systems introduce compounding uncertainty. An early mistaken assumption can propagate through dozens of later steps, and a final rationale may summarize the corrected story rather than expose the actual path. Multi-agent systems add delegation and social dynamics: one agent's summary becomes another agent's evidence. Provenance therefore needs to record not only source documents but also transformations, agent identities, model versions, tool outputs, and acceptance decisions.

The most promising design is to treat an agent episode as replayable science. Every consequential claim or action should be connected to evidence nodes and state transitions that can be re-executed or challenged. Explanations then become queries over an execution graph: which evidence supported this step, which branch was rejected, what would have happened if a tool returned a different result, and where did human approval enter? This is broader than classic XAI, but it is the natural form of explainability for autonomous systems.

Replay requires environment capture. A tool call that queried the web, a database, or a changing code repository cannot be reproduced from the agent log alone unless the relevant result is preserved or versioned. The same is true for stochastic model calls. Practical agent explainability therefore overlaps with reproducible computing: immutable inputs, snapshots, seeds where meaningful, and explicit external dependencies.

Human intervention should appear in the same graph. If a person approved a plan, corrected a source, or overrode a tool result, that event is part of the explanation of the final outcome. Treating human actions as invisible breaks the causal story and can wrongly attribute responsibility to the model. System-level XAI should make mixed human–AI agency explicit rather than pretending the agent acted alone.

The table identifies the minimum evidence needed to reconstruct an agent episode. Logging only the final natural-language answer discards most of the causal structure that an investigator will later need.

Table 5.1 — Agent explanations require evidence about the episode, not only the language model.

Concept / object	Engineering meaning
State and sources	What the system knew at the decision point: prompts, memory, retrieved documents, observations, and policy state.

Concept / object	Engineering meaning
Decision event	The model output or controller decision that selected a tool, message, plan step, or delegation.
Tool interaction	The exact call, arguments, permissions, response, and any transformation applied by surrounding software.
Environment result	What changed outside the model and which observations were returned to subsequent steps.
Replay evidence	Whether removing or changing a source, memory item, tool result, or decision changes the outcome in a controlled rerun.

Replay turns the retained trace into an explanatory instrument. If a different retrieved document leaves the action unchanged but changing a tool result reverses it, the system has learned something about the dependency structure of the episode rather than merely producing a narrative about it.

Figure 11.1. System-level explanation for an agent episode.

An AI agent may combine a foundation model with a system prompt, planner, memory, retrieval service, tools, code execution environment, policy filters, and external APIs. If the agent transfers money, edits code, or schedules a medical appointment, asking 'why did the model do that?' is underspecified. The action may have been selected by a planner, constrained by a policy, triggered by retrieved memory, or caused by a tool error.

Agent explainability should therefore be architectural. The system needs provenance across decisions: which observation entered the context, which rule or instruction had precedence, which tool was considered, which tool result changed the plan, which state variables were read and written, which safety checks ran, and which model version generated each intermediate artifact.

The diagram separates model output from environmental action. This matters because a tool can fail, an API can transform arguments, a permission layer can block a request, or a later observation can redirect the plan. The final outcome may therefore be impossible to explain from the model response alone.

Evidence chain for an agent

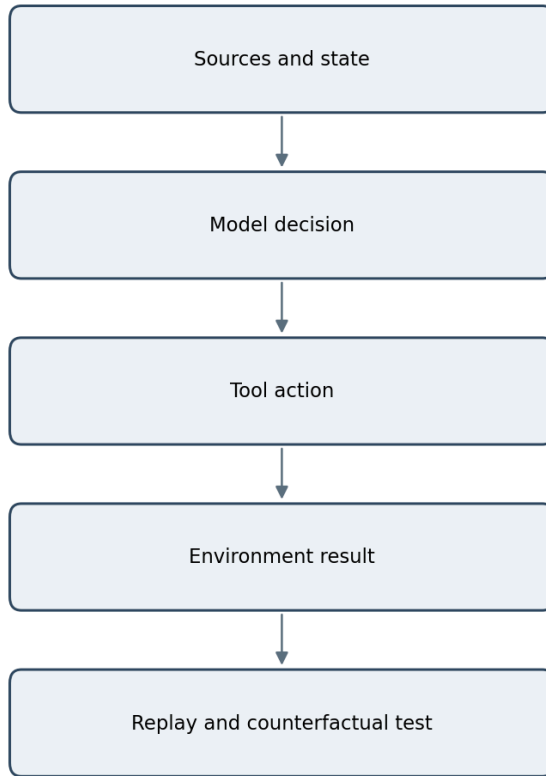


Figure 5.1 — Agent explainability follows the causal chain of an episode through sources, decisions, actions, and consequences.

Replay and counterfactual tests close the chain. By rerunning the episode with one dependency removed or altered, investigators can distinguish a merely present trace element from one that was necessary for the observed outcome.

Execution traces are necessary but not sufficient. A trace can record that the agent called a search tool and then sent an email, yet fail to explain why the email followed. Conversely, a generated summary can provide a coherent story while omitting an influential hidden state. Reliable agent explanation should combine traceability with counterfactual and causal tests.

For example, replay the episode without a particular memory item, replace a tool result with a controlled alternative, vary instruction

precedence, or block one capability. If the explanation claims that an action was triggered by a retrieved fact, removing that fact should alter the plan in predictable ways.

Long-running agents introduce temporal credit assignment. An action at time fifty may depend on an observation at time three and a summary written at time twenty. Summarization can erase provenance while preserving a conclusion. Memory retrieval can reintroduce stale or incorrect information. Tool outputs can become implicit state.

The explanation substrate should therefore preserve event lineage. Important assertions should carry source references; derived state should identify the transformations that produced it; summaries should link back to the evidence they compress. This resembles data provenance in scientific workflows more than classical local feature attribution.

In multi-agent systems, responsibility can be distributed across specialist agents, critics, planners, and verifiers. One agent may propose a plan, another reject part of it, and a coordinator synthesize the result. The explanation should represent this division of labor rather than fabricate a single-author rationale.

A useful design records proposals, objections, evidence, confidence, tool results, and final arbitration as typed artifacts. This does not guarantee correctness, but it creates an auditable causal skeleton. Disagreements between agents can themselves become explanatory signals: a decision accepted only after repeated critic failure deserves different treatment from a decision supported independently by several mechanisms.

The most promising direction is replayability. If an agent episode can be deterministically or approximately replayed from a checkpoint with controlled modifications, explanations become experimentally testable. A claim such as 'the system chose supplier B because the price limit was exceeded for supplier A' can be tested by altering only that condition and observing whether the decision changes.

For agents, a trace becomes explanatory only when replay and intervention can test which parts of the episode were necessary. Some of those properties can be strengthened further with formal verification, provided the claim is narrow enough to state precisely.

5.3 Verified explainability and bounded guarantees

Formal verification changes the grammar of an explanation. Instead of saying that sampled perturbations suggest a feature is irrelevant in a

neighborhood, a verifier may establish that within a specified region, changing that feature cannot alter the model's decision. The claim becomes stronger because it quantifies over all states represented by the formal region, subject to the soundness and completeness properties of the verifier.

The cost is scope. Neural-network verification is computationally hard, and the explanatory property must be encoded precisely. This makes formal methods most attractive for bounded, high-value claims: robustness around safety-critical inputs, invariance to a protected attribute within a region, or minimal feature sets whose fixed values guarantee a prediction. The result is not a universal explanation of the model; it is a certificate for a particular property.

FaVeX, presented at ECOOP 2026, illustrates the direction. The method accelerates verified explanation by combining batch and sequential processing and reusing information across verifier queries. The authors report meaningful formal explanations for networks with hundreds of thousands of nonlinear activations, an important scalability step. Their notion of verifier-optimal robust explanations also makes verifier incompleteness explicit rather than pretending the tool has solved an idealized optimization exactly.

Verified explanation should therefore complement, not replace, empirical and human-centered XAI. Formal proof can establish that a model behaves a certain way inside a modeled region. It cannot prove that the region represents reality, that the features carry the intended semantics, that the data are unbiased, or that a human will interpret the certificate correctly. The strongest assurance stacks use proof for the claims it can support and other evidence for the rest.

An empirical robustness test samples perturbations and observes whether an explanation changes. A formal certificate states that no perturbation inside a defined set can violate a specified property, subject to the soundness of the verifier and the formal model. This is a qualitatively different kind of evidence. It does not make the explanation semantically correct, but it can eliminate uncertainty about a precisely stated local property.

For example, a verified explanation may establish that changing a subset of nonessential features within allowed ranges cannot alter the prediction, or that a minimal set of features is sufficient to preserve an output under bounded variation. Such claims are narrow but useful because they are explicit and falsifiable.

A robust prediction is stable under perturbation. A robust explanation additionally requires that the explanatory object itself remain meaningful

under the perturbation. The two properties are related but not identical. A model can make the same prediction for many inputs while shifting which features it relies on, and an attribution map can change dramatically while the output remains stable.

Certified attribution work uses techniques such as randomized smoothing or bound propagation to place guarantees on explanation stability. These guarantees depend on a perturbation model: pixel changes, feature additions, norm-bounded input movement, or another formal set. A certificate outside its threat model says nothing.

Verified explanation methods use neural-network verifiers to establish invariances or find assignments that change a prediction. The computational cost is severe because neural-network verification has difficult worst-case complexity and an explanation can require many verifier calls. This has historically limited scale.

De Palma, Dolcetti, and Urban's ECOOP 2026 work introduces FaVeX, an algorithm that accelerates verified explanations by combining batch and sequential feature processing and reusing information from earlier verification queries. The reported experiments extend meaningful formal explanation to networks with hundreds of thousands of nonlinear activations. This is an important engineering signal: formal explanation is moving from toy demonstrations toward nontrivial networks, although it remains far from cheap verification of frontier foundation models.

Formal guarantees can certify the wrong property. If the perturbation region is unrealistic, the certificate may be irrelevant. If the feature vocabulary is semantically inappropriate, proving that a feature set is sufficient does not make the feature set human meaningful. If the predictor is trained on biased data, verified stability can preserve a biased decision perfectly.

Certification should therefore be layered. Formal verification establishes selected low-level properties. Causal validation establishes whether interventions have the intended real-world semantics. Human evaluation establishes whether the explanation supports the intended decision. Domain tests establish whether the model itself is valid in the operational regime.

A useful future direction is to attach certificates to explanation artifacts. The artifact could contain a machine-checkable statement of the property, the model hash, perturbation domain, verifier, proof object or verifier trace, and links to higher-level semantic assumptions. This would allow independent systems to verify parts of an explanation without trusting the service that produced it.

Such proof-carrying explanations resemble proof-carrying code: the consumer receives both an assertion and evidence that can be checked. For regulated or safety-critical systems, this could provide a stronger foundation than screenshot-based compliance.

The near-term opportunity is selective verification. Rather than prove an entire foundation model understandable, verify bounded components and decisions: a safety classifier, monotonic business constraint, restricted neural controller, feature invariance, or an agent policy gate. These verified islands can be embedded in larger systems and combined with empirical monitoring elsewhere.

Verified explanations are powerful because they can eliminate ambiguity about a bounded property. They are limited because most of the questions people care about are broader than the properties we can currently prove. The 2026 frontier is therefore hybrid: empirical evidence, human evaluation, semantic analysis, and selective formal guarantees.

5.4 The 2026 frontier: explanation as an assurance system

By 2026, XAI is best understood as a maturing assurance discipline rather than a competition among isolated explanation algorithms.

By 2026, the most important change in XAI is conceptual rather than algorithmic. The field is moving away from method-first thinking — choose SHAP, LIME, a saliency map, or a counterfactual — toward problem-first explanatory systems. Haufe et al. make this explicit by arguing for formalization of explanatory problems and correctness. Vinuesa, Brunton, and Mengaldo extend the ambition in another direction, presenting XAI as a way to learn from machine-learned representations in scientific discovery, optimization, and certification rather than merely justify individual predictions.

Mechanistic interpretability is also moving from artisanal inspection toward scalable tooling. SemanticLens, attribution graphs, sparse feature dictionaries, and automated semantic descriptions all try to make internal representations searchable and testable. The frontier is not settled: scale, feature validity, distributed computation, and evaluation remain open problems. What has changed is that internal explanation is increasingly treated as an experimental science of interventions rather than as visualization alone.

Multimodal and agentic systems widen the target further. The 2026 multimodal survey shows rapid expansion but limited human evaluation and public code. Agentic systems demand explanation of sequences, tool

use, and provenance. Verified explanations introduce stronger guarantees for bounded properties. Regulation and risk-management frameworks increasingly expect transparency to be connected to actual use and oversight rather than to generic disclosure.

A useful maturity map in 2026 therefore has several layers. Feature attribution and transparent tabular models are mature tools when used with appropriate semantics. Counterfactual recourse and concept-based models are established research-to-practice areas with important domain constraints. Mechanistic interpretability is a fast-moving research field with increasingly practical methods but no general solution to reverse engineering frontier models. Verified explainability is promising for selected properties but still constrained by scalability and modeling assumptions. System-level explanation for multimodal agents is emerging and likely to become one of the central engineering problems of the next few years.

A clear pattern is visible in the 2026 literature. The field is becoming more skeptical of algorithm-first XAI and more explicit about explanation correctness. Haufe and colleagues argue that popular explanation methods can systematically answer the wrong questions when feature dependencies and problem semantics are ignored. They call for well-defined tasks, objective correctness criteria, and ground-truth validation. This is less a new explainer than a change in scientific discipline.

For engineers, this is the most important state-of-the-art development because it changes how methods should be selected. LIME, SHAP, Integrated Gradients, counterfactuals, concepts, and mechanistic tools remain valuable, but each becomes an instrument inside a problem-specific validation design rather than a generic transparency layer.

Vinuesa, Brunton, and Mengaldo's August 2026 Nature Communications perspective frames explainability as a bridge from high-performing learned systems to discovery, optimization, and certification. Their key distinction between model-level explanations and system-level causal claims is especially useful. A model can expose a pattern that suggests a scientific mechanism, but independent causal reasoning and domain validation are required before the pattern becomes a claim about the physical world.

This direction expands XAI beyond responsible-AI dashboards. In scientific machine learning, interpretability may help derive candidate equations, identify invariances, reveal latent coordinates, propose

experimental interventions, or expose where learned surrogates violate known physics. Explainability becomes part of a discovery loop.

The foundation-model frontier is moving from individual neurons and heads toward learned semantic features, attribution graphs, component embeddings, and textual descriptions of activations. Sparse autoencoders address superposition by expanding dense activation spaces into sparse learned features. SemanticLens maps components into a multimodal semantic space for search and audit. Attribution graphs connect features into candidate circuits. Natural Language Autoencoders attempt to encode activations through language itself.

These approaches share a goal: find explanatory units between raw parameters and full model behavior. Their unresolved problem is validation. Learned semantic decompositions can be unstable or incomplete, and automated labels can be persuasive without being causally correct. The strongest research increasingly combines semantic description with internal interventions.

The language of the field is also changing from interpretability toward mechanistic understanding and validation. This matters. 'Understanding' should predict behavior under intervention. The SemanticLens work explicitly emphasizes component-level validation; circuit-tracing studies test pathways; Nature Machine Intelligence has highlighted the need to validate mechanistic explanations rather than stop at visualization.

The phrase 'state of the art' is often used as if it named one winning algorithm. In XAI in 2026, the more defensible interpretation is a change in engineering posture. Producing explanations is no longer the difficult part. Libraries can generate feature attributions, local surrogates, counterfactuals, concept scores, internal probes, and natural-language rationales with little friction. The frontier has moved toward specifying what those outputs mean, validating the claim they support, and integrating the evidence into systems that can be monitored and audited.

The 2026 formalization argument by Haufe and colleagues is important in this context because it attacks a root cause of the field's confusion: algorithms were often defined before explanatory problems were. Work on SemanticLens shows a complementary move toward scalable semantic validation of model components. Verified explanation research such as FaVeX demonstrates that bounded formal claims can sometimes be checked at a scale that was previously impractical. Multimodal surveys simultaneously show how far evaluation still lags behind method generation in newer domains. Together these strands do not converge on a single technique; they converge on an assurance discipline.

This perspective changes procurement and architecture decisions. A mature team should not ask a vendor whether the system 'has XAI.' It should ask which explanatory questions are supported, what the returned objects mean, how their faithfulness is tested, whether the explanation path is versioned with the model and data pipeline, how user-facing explanations differ from internal diagnostics, and which failure states are exposed. The answers reveal far more than the presence of a dashboard.

It also changes research evaluation. A new explainer should be interesting because it answers a previously hard explanatory question, improves the quality or scalability of evidence, exposes a new failure mode, or enables a stronger form of validation. A new visualization that resembles familiar XAI outputs but lacks clearer semantics or testing is less likely to move the field forward.

A mature mechanistic claim should survive alternative decompositions, prompt variations, model variants, out-of-distribution cases, and interventions designed to distinguish competing mechanisms. This standard is demanding, but without it mechanistic interpretability risks becoming a gallery of evocative internal features.

The 2026 multimodal literature is growing quickly as vision-language and multimodal foundation models spread into medicine, mobility, robotics, education, and scientific applications. A 2026 systematic survey covering 155 publications and 54 datasets finds a field still dominated by visual or quantitative techniques and notes limited public code and human evaluation. This is a warning that capability growth is outrunning evaluation maturity.

The central open problem is interaction: did the model actually combine modalities, or did one modality dominate? Explanations need to reveal fusion, substitution, contradiction handling, and cross-modal causal dependencies, not merely provide separate heat maps for image and text.

As explainability moves into production, attacks on explanation interfaces and privacy leakage become harder to ignore. A service that faithfully reveals a model can help an attacker as well as an auditor. The future XAI stack will therefore include policy-aware disclosure: different users receive different explanatory resolution, and explanation queries are logged, bounded, and monitored.

This does not weaken transparency as a value. It makes transparency an engineered capability with threat models, access control, and information budgets.

Several practices should now be considered insufficient on their own: showing one saliency map, reporting one SHAP chart without background semantics, treating attention weights as reasoning, validating

explanations only against human rationales, using one faithfulness metric, presenting chain-of-thought as direct introspection, or interpreting a classifier counterfactual as a causal recommendation.

These tools and artifacts can still be useful, but only inside a stronger explanatory design. The state of the art is the design of evidence around explanations.

The emerging state of the art is an assurance architecture rather than a single method. That observation leads naturally to the final chapter: if explanation is evidence, the next step is to make that evidence machine-checkable, replayable, and capable of being revised.

Chapter 6. Toward Executable Understanding

The mature endpoint of XAI is not a more persuasive paragraph. It is an infrastructure in which explanatory claims are connected to evidence, provenance, counter-tests, scope conditions, and explicit failure states. That idea does not eliminate ambiguity; it makes ambiguity manageable. This final chapter therefore turns explanation into an engineering object that can be stored, challenged, replayed, and updated. It develops executable explanation contracts, evidence graphs, and a reference architecture for interrogable AI. It then tests the architecture against several model families to show where the design must change. The purpose is not to declare a final theory of explainability. It is to make the next theory easier to falsify.

QUESTION TO CARRY THROUGH THE CHAPTER — How can an explanatory claim become a versioned engineering object that can be replayed, challenged, and updated?

6.1 Explanation as a scientific hypothesis

The phrase executable understanding is intentionally demanding. A natural-language explanation can be useful, but it is difficult to test automatically. An executable explanation is a claim packaged with enough structure that another process can attempt to falsify it. If the claim says a model uses a concept, the artifact links that concept to examples, a representation, an intervention, expected behavioral changes, and validation results. If the claim says an agent relied on a source, the artifact links the source to retrieval, context construction, downstream steps, and ablation tests.

This turns explanation into a scientific hypothesis rather than a rhetorical endpoint. The hypothesis can age. A new model version may break it. A broader dataset may reveal counterexamples. A stronger intervention may show that a supposedly central feature was only correlated with the real mechanism. Instead of hiding this instability, an executable explanation stores the evidence and conditions under which the claim was accepted.

Machine-readable provenance is essential. Explanations should refer to immutable model versions, datasets, prompts, source documents, transformations, feature definitions, and test results. Evidence graphs can connect a human-readable claim to the experiments that support it. Explanation contracts can act like types, preventing a local predictive attribution from being consumed by a component that expects causal recourse. Mechanism monitors can periodically retest critical explanatory relationships after retraining.

Neuro-symbolic components become useful here not because symbolic systems are inherently more truthful, but because they can express constraints and relationships that are difficult to preserve in free-form text. A system can require every consequential claim to have a source span, every recourse recommendation to satisfy feasibility rules, or every mechanistic label to be backed by an intervention test. The symbolic layer does not replace learned models; it makes the explanatory obligations explicit and executable.

The most productive conceptual shift is to treat an explanation as a hypothesis about a computational system. A hypothesis should predict observations under controlled intervention. If an attribution says a feature is necessary, remove it. If a concept is claimed to mediate a decision, intervene on the concept. If a circuit is claimed to implement a computation, ablate or patch it. If a rationale cites a retrieved source, remove the source and replay the generation.

This makes XAI iterative. An explanation is proposed, challenged, revised, compared with alternatives, and eventually rejected or supported. The workflow resembles science more than visualization.

An executable explanation pairs a claim with the code or declarative tests needed to check it. Instead of storing 'feature X was important,' the artifact could store 'under perturbation family P, masking X reduces target score by at least T in cohort C' together with a reproducible test. Instead of 'the agent chose tool A because of memory item M,' the artifact could include a replay experiment that removes M and checks the plan change.

ExecutableExplanation:

```
claim = "retrieved passage R is necessary for factual claim C"
```

```
evidence = ["retrieval_trace", "claim_source_link"]
```

```
test:
```

```
replay(without = R)
```

```
assert claim_C_is_not_produced_or_confidence_drops()
```

```
scope = "model=v24, prompt-template=p7, corpus=2026-08-30"
```

Such artifacts are valuable to humans and machines. CI systems can rerun them after model updates. Auditors can sample them. Agent supervisors can request stronger evidence when a claim lacks a corresponding test.

A future AI stack can encode explanatory semantics in types or contracts. A 'predictive attribution' object would not be implicitly convertible to a 'causal intervention' object. A 'generated rationale' would be typed differently from a 'mechanistic circuit hypothesis.' A recourse recommendation would require feasibility and causal validation metadata before it can be displayed as advice.

This may sound bureaucratic, but software engineering has repeatedly benefited from making invalid states harder to represent. Many XAI failures are category errors. A type system for explanations can prevent a dashboard or downstream agent from silently treating correlation as causation.

Treating an explanation as a hypothesis is a modest idea with radical consequences. It means the explanation can be wrong without the underlying prediction being wrong, and it can be partially right within a defined scope rather than globally true. It means the explanatory artifact should carry the evidence that supports it and the conditions under which the evidence was collected. Most importantly, it means explanation systems should preserve disagreement. If two methods support different hypotheses, the system should not automatically collapse them into one narrative merely because a user interface prefers a single answer.

This view resembles ordinary scientific work more than ordinary dashboard design. A researcher proposes a mechanism, identifies observations it predicts, designs interventions that discriminate it from alternatives, and updates confidence when new evidence arrives. XAI can follow the same pattern. The difference is that many of the experiments can be executable: perturb an input, swap a baseline, patch an activation, remove a retrieved source, replay an agent episode, or re-run a verified property after a model update.

The scientific analogy also protects against explanatory overreach. A local explanation is a local hypothesis. A concept label is a semantic hypothesis. A mechanistic circuit is a structural hypothesis. A counterfactual recourse recommendation is a hypothesis about both the model and the world. Their evidential burdens are different, but the logic is shared: state the claim, state the scope, identify a test, record the outcome, and remain explicit about what has not been established.

Complex AI systems produce decisions through networks of evidence: input observations, retrieved documents, intermediate model outputs, tool

calls, rules, policies, and learned mechanisms. A flat rationale compresses this graph into prose. A better substrate preserves the graph and lets prose be generated as a view.

An evidence graph can record which claims depend on which observations, which internal or external tests support them, where uncertainty enters, and which counterfactuals have been tried. This creates the possibility of automatically finding unsupported claims, circular justification, stale evidence, or explanations that depend on a single fragile component.

Safety engineering rarely relies on one sensor. XAI could adopt the same principle. A high-risk decision may require agreement between independent explanatory channels: behavior under perturbation, concept-level analysis, example provenance, and a mechanistic or formal test. The methods will not always agree, and disagreement is useful.

Explanatory redundancy is not majority voting over saliency maps. It means choosing methods with different failure modes. If an attribution, counterfactual, and mechanistic intervention all point toward the same dependency, confidence is stronger than if three variants of the same perturbation method agree.

If mechanistic interpretability identifies robust internal features or circuits associated with safety-relevant behavior, those mechanisms can become runtime monitors. A system could track the activation of a deceptive-planning feature, a memorization pathway, a prohibited concept, or a known shortcut and trigger deeper inspection.

This idea is speculative and technically difficult. Internal features can drift after fine-tuning, be context-dependent, and fail to capture all implementations of the behavior. Mechanism monitors should therefore be treated as additional sensors, not as perfect detectors. Their value would come from combining internal observability with behavioral and policy controls.

A further direction is to place neural models inside symbolic or declarative harnesses that represent claims, dependencies, policies, and verification obligations explicitly. The neural model proposes interpretations or hypotheses; symbolic components track provenance, enforce invariants, schedule tests, and record which claims remain unsupported.

This does not require symbolic systems to replace learned models. It uses symbolic structure where exactness is valuable: lifecycle state, evidence dependencies, causal assumptions, safety constraints, and audit logic. Neural components remain responsible for perception, semantic matching, generation, and hypothesis formation.

Foundation models increasingly help interpret other models by labeling features, summarizing attribution patterns, discovering concepts, or generating candidate experiments. This will accelerate XAI research, but it creates a recursive verification problem. The explaining model can hallucinate or project its own concepts onto the target.

The answer is not to prohibit AI-assisted interpretation but to constrain it. Let models generate hypotheses, not final truth. Require hypotheses to produce tests. Preserve raw evidence. Use independent models or symbolic checks where possible. In this role, AI becomes a scientific assistant for interpretability rather than an oracle.

A hypothesis-oriented view gives XAI a scientific posture: explanations gain strength by surviving attempts to break them. To operationalize that posture, claims and tests need a representation that software systems can retain and execute.

6.2 Machine-checkable claims, evidence graphs, and explanation contracts

The phrase executable understanding can sound grander than the engineering task actually requires. A practical starting point is much simpler: store explanatory claims in a form that software can test and invalidate.

Machine-checkable explanation does not require converting natural language into a giant formal proof. It requires separating the parts of an explanatory claim that can be represented operationally. The system can store the model version, input, background distribution, explainer configuration, referenced data, intervention, expected behavior, observed behavior, tolerance, and decision made by the evaluator. Natural language remains useful for interpretation, but the evidence underneath it becomes queryable and replayable.

An evidence graph is a convenient abstraction for this infrastructure. One node may represent a claim such as 'the alert depends materially on oxygen saturation in this case.' Other nodes represent the attribution artifact, perturbation test, model checkpoint, preprocessing version, and domain review. Edges record relations such as generated-by, tested-by, contradicts, supports, applies-to, or supersedes. The graph is not a philosophical ontology of explanation. It is an engineering ledger of why the organization believes a bounded statement about the system.

This structure makes explanatory debt visible. A claim with no counter-test is weaker than one with a successful intervention. A user-facing explanation whose evidence points to an obsolete model

version is invalid even if the prose still reads well. A mechanistic label supported by one prompt family but contradicted by another can be marked as scope-limited rather than silently reused. Versioning turns these distinctions into maintainable system state instead of institutional memory.

Explanation contracts can then act like types. A component that produces an attribution cannot automatically satisfy an interface requiring causal recourse. A source-provenance object cannot automatically satisfy an interface requiring mechanism. A verified robustness certificate can satisfy a narrow property but not a broad claim of trustworthiness. Type-like constraints prevent downstream components from consuming an explanatory object as if it carried stronger semantics than it actually does.

A practical evidence graph also needs rules for invalidation. Some dependencies are direct: if a model checkpoint changes, an attribution computed on the previous checkpoint becomes stale. Others are semantic: if the background population used by a SHAP explainer changes, the numerical artifact may still be reproducible while its interpretation no longer matches the deployed population. The lifecycle layer should therefore distinguish technical reproducibility from continued applicability.

Contradiction should be stored rather than erased. Suppose a feature-attribution test suggests that a variable is important, while a counterfactual intervention shows that realistic changes in the variable have almost no effect because correlated variables move with it. The two artifacts need not be forced into one score. They answer different questions, and their tension is itself useful evidence about the model and the data-generating process.

This table is deliberately modest. It does not attempt to encode the full meaning of an explanation in a formal language. It encodes enough structure to make the evidence reproducible and to prevent unsupported semantic upgrades.

Table 6.1 — An executable explanation is a claim connected to replayable evidence.

Concept / object	Engineering meaning
Claim	The bounded statement being asserted, written so that its scope and explanatory target are explicit.
Provenance	The model, data, prompt, explainer, configuration, and runtime state that produced the supporting artifact.

Concept / object	Engineering meaning
Test	A perturbation, counterfactual, intervention, user study, or formal check that can count for or against the claim.
Result and tolerance	The observed behavior, acceptance threshold, uncertainty, and any known counterexamples.
Lifecycle state	Whether the claim is accepted, contradicted, scope-limited, stale after an update, or awaiting additional evidence.

Once these fields exist, explanations can participate in engineering workflows. A model update can invalidate stale claims automatically, regression tests can be rerun, and oversight tools can show not only what the system currently believes about itself but why that belief exists.

The evidence store also gives organizations a principled way to manage explanation drift. When preprocessing, retrieval, prompt templates, feature dictionaries, or external tools change, dependency edges identify which claims require re-evaluation. This is more precise than scheduling a generic annual review because the system knows which explanatory statements are actually downstream of the changed component.

Access control matters because evidence graphs can contain sensitive material: exact prompts, personal records, training examples, proprietary model internals, or security-relevant failure modes. The graph should therefore separate the existence of a claim from the audience allowed to inspect every supporting artifact. A public-facing explanation may expose a bounded summary while internal auditors retain richer diagnostic evidence under stricter permissions.

Finally, machine-checkable explanations should record abstention. If the system cannot construct an intervention that stays within the validated domain, or if competing methods disagree beyond an accepted tolerance, the correct lifecycle state may be unresolved rather than accepted. Encoding unresolved states is a feature, not a defect: it prevents the infrastructure from manufacturing certainty merely because a downstream interface expects a single answer.

The same machinery supports disagreement and revision. Two explanation artifacts can support competing claims about a failure. The system can attach new tests rather than forcing immediate resolution. When a model is retrained, affected claims can be marked stale and re-evaluated. This is closer to continuous integration than to static

documentation, and it is a more realistic way to manage explainability in systems that change frequently.

Machine-checkable evidence does not replace interpretation; it gives interpretation a substrate that can be audited. The next section assembles those pieces into a reference architecture for systems designed to be interrogated from the beginning.

6.3 A reference architecture for interrogable AI

If explanation is evidence, the system needs an architecture that preserves the evidence before an incident occurs.

A system designed to be interrogated should separate five concerns. The first is prediction or generation: the model that performs the task. The second is observability: logs, activations where available, retrieval traces, tool calls, uncertainty, and lineage. The third is explanatory analysis: attribution, counterfactuals, concepts, data influence, probes, and mechanistic tests. The fourth is validation: sanity checks, perturbation experiments, formal verification for selected properties, human evaluation, and regression tests. The fifth is presentation and governance: audience-specific views, review workflows, retention, access control, and incident response.

Keeping these layers separate prevents one of the most common architectural mistakes: using a generated explanation as both the evidence and the interface. In the reference design, generated language is a renderer over structured evidence. A user can read a concise narrative, while an auditor can drill down into the underlying tests. An engineer can replay a failure. A policy rule can reject an explanation whose evidence is stale or whose contract does not match the requested decision.

The minimum viable XAI review is correspondingly simple. Before deployment, state the explanatory questions. Choose methods whose semantics match those questions. Establish at least one falsification test for each important method. Evaluate representative and difficult cases, not only attractive examples. Record method configuration and provenance. Test whether explanations remain useful under realistic distribution shift. Decide what will be monitored after deployment and what evidence must be retained for incident analysis.

The deepest design rule of this book is that explainability should reduce uncertainty about a system, not merely produce a story about it. Sometimes the right output is a feature attribution. Sometimes it is a counterfactual. Sometimes it is a formal certificate, a replay trace, a causal intervention, or a warning that no reliable explanation is available.

Systems become trustworthy not when they can always answer “why?” in fluent language, but when they can distinguish what is known, what is inferred, what has been tested, and what remains unresolved.

Figure 6.1. A reference architecture for interrogable AI systems.

Most deployed AI is trusted by performance: the system works on a benchmark, passes validation, and is monitored after deployment. Explainability offers a path toward trust by partial understanding, where engineers know something about why the model works and what mechanisms fail. Complete understanding is unrealistic for many learned systems, but partial mechanistic knowledge can still materially improve assurance.

The future is therefore hybrid. Empirical performance remains essential. Causal and mechanistic explanations add insight. Formal verification certifies selected properties. Provenance and replay support audit. Human-centered interfaces translate evidence into decisions. No layer is sufficient alone.

Software systems are easier to debug when they expose logs, metrics, traces, state transitions, and invariants. AI systems need analogous observability. Model inputs and preprocessing should be traceable. Important intermediate representations should be accessible where feasible. Retrieval should preserve source lineage. Agents should record tool choices and state updates. Explanation methods should be versioned and testable.

The architecture prevents one component from becoming both advocate and judge of its own explanation. An analyzer can generate a hypothesis quickly; a different validation layer decides whether the hypothesis survives the relevant tests. Provenance connects both to the deployed system state.

Stack for interrogable AI systems

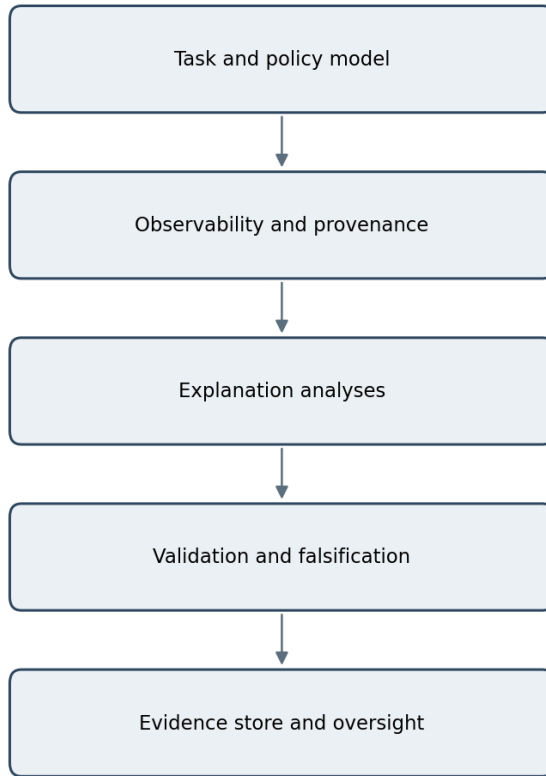


Figure 6.1 — Interrogable AI separates observability, explanation generation, validation, and evidence retention into distinct layers.

The evidence store then gives the organization memory. Claims can be traced across model updates, contradicted by new experiments, or restricted to narrower scopes. This is the key difference between a dashboard and an explanatory infrastructure.

This does not mean logging every activation in every request. Observability is selective. The architecture should identify points where explanatory evidence has high value: high-risk decisions, anomalous predictions, policy transitions, uncertain actions, and incidents.

A compact selection rule is useful. If the question is 'what locally changes the prediction?', use sensitivity or perturbation-based attribution and validate it. If the question is 'what pattern does the model use globally?', prefer intrinsic structure, global surrogates, response curves,

cohort analyses, or concept audits. If the question is 'what can the user do?', use recourse with feasibility and causal modeling. If the question is 'where did this behavior come from?', use training-data attribution and provenance. If the question is 'how did the network compute this?', use internal interventions and mechanistic analysis.

For LLMs, distinguish generated rationales from input attribution and mechanistic evidence. For agents, start with execution provenance and replay. For safety properties, consider formal verification when the property can be stated narrowly enough.

A practical explainability architecture can be divided into five services. The observation layer records inputs, outputs, state, and provenance. The explanation layer generates task-specific artifacts such as attributions, concepts, counterfactuals, or traces. The validation layer runs sanity tests, counterfactual replays, robustness checks, and formal verification where applicable. The evidence layer stores explanation objects and their dependencies. The presentation layer renders role-specific views for engineers, auditors, operators, and affected users.

These services should not be coupled to one explainer library. Explanation methods evolve quickly. The stable interface should be the explanatory contract and evidence schema.

Before deployment, a team should be able to answer a small set of questions in prose. What decision or behavior requires explanation? Who will use the explanation? Which explanatory quantity is being estimated? What assumptions does the method make about missing, changed, or dependent features? What evidence demonstrates faithfulness? Which perturbations or alternative explanations were tested? How does the explanation behave across relevant groups and distribution shifts? What information does the interface leak? How is the explanation reproduced after the model changes?

An interrogable AI architecture begins with observability. If the system does not retain the relevant inputs, model and prompt versions, retrieved sources, transformations, tool calls, timing, and outputs, later explanation will reconstruct a story from missing evidence. Observability is therefore not an optional logging concern added after the XAI method. It determines which explanatory questions remain answerable after an incident.

Above observability sits a family of explanation analyses rather than one universal explainer. Tabular models may expose additive effects and counterfactuals. Vision models may require spatial perturbations, prototypes, or concept tests. Foundation models may require prompt counterfactuals, source ablations, probes, and activation interventions. Agentic systems require episode reconstruction and replay. Each analyzer

should return both its artifact and machine-readable metadata describing semantics, assumptions, version dependencies, and known invalid regimes.

Validation is a separate layer because no explanation method should be trusted by construction. The validation service runs sanity checks, perturbation experiments, stability tests, subgroup analysis, causal interventions, user studies where appropriate, and formal verification for bounded properties. The key design choice is separation of concerns: generating an explanation and testing an explanation are different capabilities and may intentionally use different algorithms or models.

The evidence store binds the layers together. It records claims, artifacts, tests, provenance, reviewer decisions, and scope conditions. The store makes explanations longitudinal: a team can ask what evidence supported a claim three model versions ago, whether the same mechanism still appears, or which user-facing explanations became stale after a preprocessing change. This historical dimension is essential for auditing and for scientific learning from deployment.

Oversight sits above the evidence rather than replacing it. Human reviewers need interfaces that present the claim first, then the strongest supporting and contradicting evidence, then technical detail on demand. Automated agents can help by finding anomalies, generating counter-tests, or summarizing evidence, but their outputs should enter the same evidence system rather than becoming an untracked meta-layer. In this architecture even the explainer can be explained, because its configuration and validation history are first-class objects.

The architecture also supports selective rigor. Not every explanation needs a theorem prover, and not every decision needs a mechanistic circuit. Low-risk debugging may accept fast heuristic evidence. High-stakes recourse may require feasibility constraints, causal review, and regression testing. Safety-critical bounded properties may justify formal verification. The system should make that gradient explicit so that rigor is allocated according to consequence rather than according to fashion.

Finally, interrogability must be designed before deployment. Retrofitting it later is expensive because information has already been discarded, runtime context has not been versioned, and users have learned to depend on explanation formats whose semantics were never specified. The cheapest moment to define an explanation contract is when the model interface and data lineage are being designed, not when the first complaint arrives.

This is intentionally more demanding than 'we use SHAP.' A named algorithm is an implementation detail; assurance is a system property.

A strong first exercise for students and engineering teams is to attempt to break an attribution method on a model they control. Train a small model with a known shortcut feature. Produce explanations with two different methods. Randomize the model weights and check whether the explanations change. Replace the background distribution and observe how attributions move. Remove top-ranked features and compare with random removal. Then retrain after removal to control for distribution shift. The objective is not to crown a winner but to experience how explanation claims depend on experimental design.

Build a simple tabular classifier and generate nearest counterfactuals. Then add immutable-feature constraints, action costs, and causal dependencies. Observe how the recommended changes evolve. Finally, perturb the model parameters or retrain on a shifted sample and test whether the recommendation remains valid. The exercise makes the gap between boundary explanation and real-world action concrete.

Choose a language-model task with a controlled hint. Ask the model for a rationale and record whether the hint is mentioned. Then remove or reverse the hint and measure answer changes. Attribute context spans if the model interface allows it. If weights are available, probe selected layers or perform activation interventions using an interpretability framework. Compare what each level of evidence can legitimately claim.

Construct a small tool-using agent with retrieval and memory. Record a full execution trace. Generate a natural-language explanation of one action, then replay the episode after deleting a memory item, changing a tool response, or modifying one instruction. Convert the explanation into an executable claim that predicts the replay result. This exercise reveals how quickly agent explainability becomes systems engineering rather than model visualization.

Explainability is not one property and no method is universally best. Intrinsic interpretability should be preferred when it delivers the needed capability. Post-hoc methods are measurements whose semantics depend on assumptions. Plausibility is not faithfulness. Predictive dependence is not causation. Counterfactual endpoints are not automatically actions. LLM rationales are not privileged introspection. Mechanistic interpretations are hypotheses until interventions validate them. Formal certificates are strong only for the property they actually prove.

The most durable engineering idea is simpler: build AI systems that can be interrogated. Preserve provenance. Formulate explicit questions.

Generate explanations appropriate to those questions. Try to falsify them. Store the evidence. Re-run the tests when the system changes. Explainability then becomes part of the discipline by which machine learning moves from impressive prediction toward dependable engineering.

Final engineering rule. An explanation earns trust by surviving attempts to prove it wrong.

The architecture is intentionally modular because explanatory needs change across domains. The final section tests the design against model families where the meaning of an intervention changes fundamentally.

6.4 How the design changes across time series, graphs, reinforcement learning, recommenders, and scientific AI

A reference architecture is only useful if it survives contact with domains whose notion of a valid intervention is radically different.

Time-series models make temporal structure part of the explanation. A feature is not merely 'heart rate' or 'temperature'; it is a value at a time, a trend, a lag, a season, or an interaction across windows. Attribution methods that independently perturb timestamps can create impossible histories. A meaningful explanation therefore needs interventions that respect temporal continuity and the data-generation process. Global analyses should also distinguish stable seasonal effects from transient operational regimes, because a model can be faithful in one regime and misleading in another.

Graph neural networks add relational dependence. The prediction for one node may depend on neighbors, paths, edge types, and learned message passing. Explanations can select subgraphs, edges, or node features, but the selected structure is only meaningful if the intervention preserves graph semantics. Removing an edge from a molecule, a social network, and a knowledge graph represents three very different hypothetical worlds. A graph explanation should therefore state what graph edit is being modeled and whether the edited graph remains valid in the domain.

Reinforcement learning changes the target again because an action is valuable through its consequences over time. Explaining a single action may require the policy state, expected future reward, uncertainty, and alternatives that were available. For learned policies embedded in environments, replay and counterfactual simulation are often more informative than static saliency. If the environment is partially observed,

an explanation should also distinguish what the agent inferred from what was actually observed.

Recommender and ranking systems are inherently comparative. The practical question is rarely why one score is high in isolation; it is why item A appeared above item B, which signals dominate exposure, and how the ranking changes when context or user history changes. Explanations that ignore the candidate set can be misleading because ranking is relative. Long feedback loops also matter: recommendations change behavior, behavior changes training data, and future explanations inherit the resulting selection effects.

Scientific machine learning raises the evidential bar because the goal may be discovery rather than only prediction. An explanation that identifies a feature correlation can suggest a hypothesis, but scientific interpretation requires domain constraints, causal reasoning, and independent validation. Here XAI is most useful when it helps formulate experiments: identify a candidate variable, show where the model's behavior changes, propose a controlled perturbation or simulation, and then compare the result with theory or new data. The explanation becomes part of a discovery workflow rather than a final answer.

Across these domains, the common lesson is that explanation semantics come from the task and the allowed intervention, not from the visualization. Time-series explanations require temporally coherent changes; graph explanations require valid graph edits; reinforcement learning requires trajectory-level reasoning; ranking requires comparative context; scientific AI requires experiments that connect learned structure to the world. A reusable XAI platform therefore needs a stable evidence architecture with domain-specific intervention models, not one universal explanation widget.

The durable principle is therefore not a preferred explainer. It is the discipline of matching an explanatory claim to a valid intervention, preserving the evidence, and designing the system so that future observers can challenge what the present team believes.

Appendix A. Student Glossary and Background Links

This appendix explains terms that a technically curious reader may encounter before taking a dedicated course in machine learning, statistics, causal inference, or software assurance. The definitions are intentionally operational rather than exhaustive. Each row includes a Wikipedia background link as a starting point for general orientation; the research literature in the bibliography should be preferred for technical claims about XAI.

Table A.1 — Background glossary for readers entering XAI from general software engineering or secondary-school mathematics.

Term	Plain-language explanation and background link
Artificial intelligence	The broad field of building systems that perform tasks associated with perception, reasoning, language, planning, or decision making. In this book the term includes learned models and the larger software systems around them. Wikipedia
Machine learning	A family of methods in which behavior is learned from data or interaction rather than being fully programmed as explicit rules. Wikipedia
Supervised learning	Learning from examples paired with target labels or values, such as images with diagnoses or applications with historical outcomes. Wikipedia
Classification	Predicting one of several categories, often together with a score or probability. Many XAI examples use classification because a single decision is easy to localize. Wikipedia
Regression	Predicting a numerical quantity rather than a discrete class. Explanations must then refer to changes in the predicted value rather than class changes alone. Wikipedia

Term	Plain-language explanation and background link
Feature	An input variable or derived representation used by a model. A feature can be a table column, image region, token representation, sensor summary, or learned internal direction. Wikipedia
Training data	Examples used to fit model parameters. Data-attribution methods ask how particular training examples or groups influenced later behavior. Wikipedia
Validation set	Data held out from training and used to tune or select a model. It is distinct from explanation validation, which tests claims about model behavior. Wikipedia
Test set	Held-out data used to estimate how a model performs on unseen examples. A test set alone rarely establishes whether an explanation is faithful. Wikipedia
AUROC	Area under the receiver operating characteristic curve, a common measure of ranking quality for binary classifiers. Good AUROC does not imply that a model is explainable or safe. Wikipedia
Linear model	A model whose prediction is built from weighted input terms. Linear structure can be directly inspectable when variables are meaningful and interactions are limited. Wikipedia
Decision tree	A branching prediction structure based on sequential conditions. Small trees can be readable; large ensembles are usually not transparent as a whole. Wikipedia
Generalized additive model	A model that combines learned one-dimensional response functions, often providing a useful compromise between nonlinear accuracy and inspectable structure. Wikipedia

Term	Plain-language explanation and background link
Gradient boosting	A technique that builds a strong predictor by adding many weak models sequentially. Explainable Boosting Machines use related ideas with an interpretable additive structure. Wikipedia
Neural network	A parameterized computational system built from layers of transformations. Modern deep networks can learn rich representations but are usually not directly interpretable. Wikipedia
Gradient	A quantity describing how a function changes with respect to its inputs or parameters. Gradient-based XAI methods use this local sensitivity information. Wikipedia
Backpropagation	The standard algorithmic procedure used to compute gradients through layered neural networks. Some attribution methods reuse the same computational machinery. Wikipedia
Sensitivity analysis	The study of how outputs change when inputs or assumptions change. Much of attribution and robustness testing can be understood as specialized sensitivity analysis. Wikipedia
Baseline	A reference input or distribution used to define what counts as absence, replacement, or comparison in an explanation. Changing the baseline can change the meaning of the result. Wikipedia
Perturbation	A controlled change to an input, internal state, or environment used to test whether the model response changes as an explanatory claim predicts. Wikipedia

Term	Plain-language explanation and background link
Shapley value	A cooperative-game-theory allocation rule that distributes a total value among contributors. SHAP adapts this idea to model predictions under a chosen background and missingness convention. Wikipedia
LIME	A local surrogate approach that approximates model behavior around one example using perturbed samples. Its meaning depends on how that local neighborhood is generated. Wikipedia
SHAP	A family of Shapley-style explanation methods that allocate a prediction relative to a reference distribution. SHAP is not automatically a causal explanation. Wikipedia
Counterfactual	A statement about what would happen under a different condition. In XAI, counterfactuals often ask which input changes would alter a prediction. Wikipedia
Causal inference	Methods for reasoning about the effects of interventions rather than only correlations. Causal assumptions become essential when explanations are used to recommend actions. Wikipedia
Causal model	A representation of assumed causal relationships among variables. Structural causal models are frequently used to formalize interventions and counterfactual reasoning. Wikipedia
Algorithmic recourse	A family of methods for finding feasible actions that could change an unfavorable automated decision. Recourse is stronger than merely finding an editable counterfactual input. Wikipedia

Term	Plain-language explanation and background link
Calibration	Agreement between predicted confidence and observed frequency. Human-facing explanations should help users calibrate reliance rather than simply increase trust. Wikipedia
Distribution shift	A change between the data distribution used during development and the distribution encountered later. Explanation quality can drift under the same changes that affect prediction quality. Wikipedia
Concept drift	A change over time in the relationship between inputs and the target. It can make both a predictor and its established explanations stale. Wikipedia
Adversarial example	An input deliberately modified to induce unwanted model behavior. Explanation interfaces can also be attacked or manipulated and should be treated as a security surface. Wikipedia
Differential privacy	A mathematical framework for limiting how much information about individual records can be inferred from a computation. Explanation services may need privacy controls because they expose additional information about models and data. Wikipedia
Algorithmic fairness	The study of how automated decisions distribute errors, benefits, and burdens across people or groups. Explanation can support fairness audits but does not itself guarantee fairness. Wikipedia
Human in the loop	A system design in which people remain involved in decisions, supervision, or correction. Useful explanation should improve appropriate oversight rather than merely make automation feel understandable. Wikipedia

Term	Plain-language explanation and background link
Automation bias	The tendency to over-rely on automated recommendations. Fluent or persuasive explanations can increase this risk if they hide uncertainty or failure modes. Wikipedia
Large language model	A neural language model trained at large scale to predict and generate text, often extended to code, tools, images, or other modalities. Wikipedia
Transformer	The dominant neural architecture behind modern language models and many multimodal systems, based heavily on attention and learned representations. Wikipedia
Attention	A mechanism that computes context-dependent weighted interactions among representations. Attention weights can be useful diagnostics but are not automatically explanations of a model decision. Wikipedia
Chain-of-thought prompting	Prompting a language model to produce intermediate reasoning-like text. The generated text is observable behavior and may be useful, but it is not guaranteed to be a faithful trace of hidden computation. Wikipedia
Representation learning	Learning internal features from data instead of specifying all features manually. Probes and mechanistic methods study what information those representations contain and use. Wikipedia
Probe	A simple model trained on internal activations to test whether information about a property is decodable. Decodability alone does not prove that the original model causally uses that information. Wikipedia

Term	Plain-language explanation and background link
Activation patching	A causal-intervention technique in which internal activations are replaced with values from another run to test whether a component influences behavior. Wikipedia
Sparse autoencoder	An autoencoder trained to represent activations with sparse features. Such features can be easier to label, but labels still require validation. Wikipedia
Sparse coding	Representing a signal with a relatively small number of active components from a larger dictionary. Sparse feature dictionaries are used in modern mechanistic interpretability. Wikipedia
Mechanistic interpretability	Research that seeks structured, causally testable accounts of how internal model components implement behavior. Wikipedia
Multimodal learning	Learning across more than one modality, such as text and images. Explanation then has to address how evidence is aligned and combined across representations. Wikipedia
Intelligent agent	A system that observes a state or environment and takes actions toward objectives. Agent explanations must often reconstruct sequences of decisions and consequences. Wikipedia
Software agent	A software component that acts with some autonomy, often invoking tools or services. Modern AI agents combine language models with software-agent infrastructure. Wikipedia
Data provenance	Information about where data came from, how it was transformed, and which process produced an artifact. Provenance is foundational for reproducible system-level explanations. Wikipedia

Term	Plain-language explanation and background link
Formal verification	Techniques that mathematically prove whether a system satisfies a precisely stated property under a model of its behavior. Wikipedia
Model checking	A formal-verification technique that explores a state model to determine whether specified properties hold. It is relevant when explanatory properties can be expressed in a tractable form. Wikipedia
Knowledge graph	A graph representation of entities and relationships. Evidence graphs proposed in this book use a similar structure to connect explanatory claims, artifacts, tests, and provenance. Wikipedia
Graph neural network	A neural model designed for graph-structured data. Meaningful explanation often requires valid interventions on nodes, edges, or subgraphs. Wikipedia
Reinforcement learning	Learning to choose actions through interaction and reward. Explanations may need to cover trajectories and future consequences rather than isolated predictions. Wikipedia
Recommender system	A system that ranks or selects items for users. Explanations are often comparative because the meaningful question is why one item was ranked above another. Wikipedia
Time series	Data ordered in time. Explanation methods must preserve temporal coherence when perturbing or comparing values. Wikipedia

Term	Plain-language explanation and background link
Model card	A structured document describing intended use, evaluation, limitations, and other information about a model. Explanation evidence can complement, but should not be confused with, documentation. Wikipedia
Out-of-distribution detection	Methods for identifying inputs that differ materially from the data regime on which a model was validated. Sometimes the best explanation is that the system is outside its known operating region. Wikipedia

The glossary is intentionally not a substitute for the bibliography. Wikipedia is used here as a low-friction orientation layer for prerequisite concepts. Claims about XAI methods, evaluation, regulation, and the 2025–2026 research frontier should be traced to the original papers and institutional sources listed below.

Appendix B. Method Selection

Notes

Method selection should begin from the explanatory question and evidence requirement, not from familiarity with a library. The table below compresses the book into a practical first-pass guide. It does not declare winners; it identifies where each family of methods is useful and which misunderstanding should be ruled out first.

The guide is most useful when read from left to right: identify the explanatory object, then identify the first semantic mistake that could invalidate its use. This keeps method selection tied to the failure mode that matters most.

Table B.1 — A compact method-selection guide.

Concept / object	Engineering meaning
Transparent model	Use when competitive performance is possible with inspectable structure. First risk: confusing visibility with validity or causality.
Gradient / Integrated Gradients	Use for local sensitivity or path-based attribution in differentiable models. First risk: baseline semantics, saturation, and local sensitivity being described as causal importance.
Perturbation / occlusion	Use when a meaningful input intervention can be defined. First risk: creating out-of-distribution samples that test the perturbation mechanism more than the model.
LIME	Use for local surrogate reasoning. First risk: an artificial neighborhood that does not resemble the data regime relevant to the decision.
SHAP	Use for additive allocation relative to an explicit background and missingness convention. First risk: interpreting allocation as causal responsibility.
Counterfactual	Use for contrastive questions about how a prediction changes. First risk: infeasible or immutable changes.

Concept / object	Engineering meaning
Recourse	Use when the system must suggest actionable interventions. First risk: advice that ignores causal dependencies, cost, strategic response, or model updates.
Concept / prototype methods	Use when experts reason in semantic categories or representative cases. First risk: assuming that a human label gives ground-truth meaning to an internal feature.
Data attribution	Use when training examples, sources, or datasets may explain a behavior. First risk: confusing influence under one approximation with a complete causal account of training.
Mechanistic interpretability	Use when internal computation itself is the explanatory target. First risk: treating a convenient feature dictionary or circuit graph as a unique decomposition of the network.
Agent replay	Use when behavior unfolds through tools, memory, retrieval, and environment state. First risk: retaining a trace without testing which elements were necessary.
Formal verification	Use for narrow, high-consequence properties that can be stated precisely and checked tractably. First risk: proving a property that is rigorous but irrelevant to the real explanatory question.

In mature systems several rows will be used together. A user-facing counterfactual may depend on an interpretable structural model, a causal feasibility layer, a robustness test, and a provenance record. XAI therefore becomes compositional as soon as the consequence of the explanation exceeds simple debugging.

References and Recommended Reading

The bibliography combines foundational methods, evaluation work, regulation and assurance sources, and selected 2025–2026 research that marks the present frontier. Original papers are preferred for technical claims. Wikipedia links used in Appendix A are repeated afterward as background orientation rather than as primary scholarly evidence.

- Doshi-Velez, F., and Kim, B. (2017). Towards a Rigorous Science of Interpretable Machine Learning. arXiv:1702.08608.
- Guidotti, R., Monreale, A., Ruggieri, S., Turini, F., Giannotti, F., and Pedreschi, D. (2018). A Survey of Methods for Explaining Black Box Models. *ACM Computing Surveys* 51(5). [DOI](#).
- Murdoch, W. J., Singh, C., Kumbier, K., Abbasi-Asl, R., and Yu, B. (2019). Definitions, methods, and applications in interpretable machine learning. *PNAS* 116(44). [DOI](#).
- Rudin, C. (2019). Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence* 1, 206–215. [DOI](#).
- Ribeiro, M. T., Singh, S., and Guestrin, C. (2016). Why Should I Trust You? Explaining the Predictions of Any Classifier. *KDD 2016*. [DOI](#).
- Lundberg, S. M., and Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. *NeurIPS 2017*.
- Sundararajan, M., Taly, A., and Yan, Q. (2017). Axiomatic Attribution for Deep Networks. *ICML 2017, PMLR 70*.
- Selvaraju, R. R. et al. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. *ICCV 2017*.
- Ribeiro, M. T., Singh, S., and Guestrin, C. (2018). Anchors: High-Precision Model-Agnostic Explanations. *AAAI 2018*.
- Kim, B. et al. (2018). Interpretability Beyond Feature Attribution: Quantitative Testing with Concept Activation Vectors (TCAV). *ICML 2018*.
- Koh, P. W., and Liang, P. (2017). Understanding Black-box Predictions via Influence Functions. *ICML 2017*.
- Chen, C. et al. (2019). This Looks Like That: Deep Learning for Interpretable Image Recognition. *NeurIPS 2019*.
- Wachter, S., Mittelstadt, B., and Russell, C. (2017/2018). Counterfactual Explanations without Opening the Black Box: Automated Decisions and the GDPR. *Harvard Journal of Law & Technology* 31. arXiv:1711.00399.
- Karimi, A.-H., Schölkopf, B., and Valera, I. (2021). Algorithmic Recourse: from Counterfactual Explanations to Interventions. *FACCT 2021*.

- Heskes, T., Sijben, E., Bucur, I. G., and Claassen, T. (2020). Causal Shapley Values. NeurIPS 2020.
- Koh, P. W. et al. (2020). Concept Bottleneck Models. ICML 2020.
- Kim, E. et al. (2023). Probabilistic Concept Bottleneck Models. ICML 2023.
- Hu, L. et al. (2025). Editable Concept Bottleneck Models. ICML 2025.
- Adebayo, J. et al. (2018). Sanity Checks for Saliency Maps. NeurIPS 2018.
- Hooker, S., Erhan, D., Kindermans, P.-J., and Kim, B. (2019). A Benchmark for Interpretability Methods in Deep Neural Networks. NeurIPS 2019.
- Yeh, C.-K. et al. (2019). On the (In)fidelity and Sensitivity of Explanations. NeurIPS 2019.
- DeYoung, J. et al. (2020). ERASER: A Benchmark to Evaluate Rationalized NLP Models. ACL 2020.
- Arras, L., Osman, A., and Samek, W. (2022). CLEVR-XAI: A benchmark dataset for ground-truth evaluation of neural network explanations. Information Fusion 81.
- Hesse, R., Schaub-Meyer, S., and Roth, S. (2023). FunnyBirds: A Synthetic Vision Dataset for a Part-Based Analysis of Explainable AI Methods. ICCV 2023.
- Agarwal, C. et al. (2022). OpenXAI: Towards a Transparent Evaluation of Post hoc Model Explanations. NeurIPS Datasets and Benchmarks 2022.
- Hedström, A. et al. (2023). Quantus: An Explainable AI Toolkit for Responsible Evaluation of Neural Network Explanations and Beyond. JMLR 24(34), 1–11.
- Jahn, T., Hühn, P., and Reichert, M. (2026). A review on empirical studies in explainable artificial intelligence. Artificial Intelligence Review 59, 175.
- Slack, D. et al. (2020). Fooling LIME and SHAP: Adversarial Attacks on Post hoc Explanation Methods. AIES 2020.
- Zhao, H. et al. (2024). Explainability for Large Language Models: A Survey. ACM Transactions on Intelligent Systems and Technology. [DOI](#).
- Turpin, M. et al. (2023). Language Models Don't Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting. NeurIPS 2023.
- Cornish, C., and Rogers, A. (2025). Examining the Faithfulness of Deepseek R1's Chain-of-Thought Reasoning. CHOMPS 2025 / ACL Anthology.
- Meng, K. et al. (2022). Locating and Editing Factual Associations in GPT. NeurIPS 2022.
- Ghandeharioun, A. et al. (2024). Patchscopes: A Unifying Framework for Inspecting Hidden Representations of Language Models. ICML 2024.
- Cunningham, H. et al. (2024). Sparse Autoencoders Find Highly Interpretable Features in Language Models. ICLR 2024.
- Bricken, T. et al. (2023). Towards Monosemanticity: Decomposing Language Models With Dictionary Learning. Transformer Circuits Thread.
- Templeton, A. et al. (2024). Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet. Transformer Circuits Thread.

- Shu, D. et al. (2025). A Survey on Sparse Autoencoders: Interpreting the Internal Mechanisms of Large Language Models. Findings of EMNLP 2025. [DOI](#).
- Lindsey, J. et al. / Anthropic interpretability team (2025). Circuit Tracing and On the Biology of a Large Language Model. Transformer Circuits Thread, March 2025.
- Anthropic interpretability team (2025). Open-sourcing circuit-tracing tools. 29 May 2025.
- Dreyer, M. et al. (2025). Mechanistic understanding and validation of large AI models with SemanticLens. Nature Machine Intelligence 7, 1572–1585. [DOI](#).
- Anthropic interpretability team (2026). Natural Language Autoencoders Produce Unsupervised Explanations of LLM Activations. Transformer Circuits Thread, May 2026.
- De Palma, A., Dolcetti, G., and Urban, C. (2026). Faster Verified Explanations for Neural Networks. ECOOP 2026, LIPIcs 372:3. [DOI](#).
- Haufe, S. et al. (2026). Explainable AI needs formalization. npj Artificial Intelligence 2, 42. [DOI](#).
- Vinuesa, R., Brunton, S. L., and Mengaldo, G. (2026). Explainable AI: learning from the learners. Nature Communications 17, 7933. [DOI](#).
- Giovannoni, C. et al. (2026). A survey on multimodal explainable Artificial Intelligence. Intelligent Systems with Applications 31, 200671. [DOI](#).
- Wilkinson, C. et al. (2026). Explaining explainability: A comprehensive survey on explainable artificial intelligence and relevant industry applications. Intelligent Systems with Applications 30, 200647. [DOI](#).
- European Union (2024, consolidated 2026). Regulation (EU) 2024/1689, Artificial Intelligence Act, especially Articles 13 and 14. EUR-Lex.
- Tabassi, E. (2023). Artificial Intelligence Risk Management Framework (AI RMF 1.0). NIST AI 100-1. [DOI](#).
- NIST (2026). AI Risk Management Framework resources and concept note for a Trustworthy AI in Critical Infrastructure profile.
- SHAP documentation (current through 2026). [Documentation](#).
- Captum documentation (current through 2026). [Documentation](#).
- InterpretML documentation (current through 2026). [Documentation](#).
- Quantus project and documentation. [Project](#).
- OpenXAI project and leaderboard. [Project](#).

Background links used in Appendix A

- Artificial intelligence. [Wikipedia](#)
- Machine learning. [Wikipedia](#)
- Supervised learning. [Wikipedia](#)
- Classification. [Wikipedia](#)

Regression. [Wikipedia](#)
Feature. [Wikipedia](#)
Training data. [Wikipedia](#)
Validation set. [Wikipedia](#)
Test set. [Wikipedia](#)
AUROC. [Wikipedia](#)
Linear model. [Wikipedia](#)
Decision tree. [Wikipedia](#)
Generalized additive model. [Wikipedia](#)
Gradient boosting. [Wikipedia](#)
Neural network. [Wikipedia](#)
Gradient. [Wikipedia](#)
Backpropagation. [Wikipedia](#)
Sensitivity analysis. [Wikipedia](#)
Baseline. [Wikipedia](#)
Perturbation. [Wikipedia](#)
Shapley value. [Wikipedia](#)
LIME. [Wikipedia](#)
SHAP. [Wikipedia](#)
Counterfactual. [Wikipedia](#)
Causal inference. [Wikipedia](#)
Causal model. [Wikipedia](#)
Algorithmic recourse. [Wikipedia](#)
Calibration. [Wikipedia](#)
Distribution shift. [Wikipedia](#)
Concept drift. [Wikipedia](#)
Adversarial example. [Wikipedia](#)
Differential privacy. [Wikipedia](#)
Algorithmic fairness. [Wikipedia](#)
Human in the loop. [Wikipedia](#)
Automation bias. [Wikipedia](#)
Large language model. [Wikipedia](#)
Transformer. [Wikipedia](#)
Attention. [Wikipedia](#)
Chain-of-thought prompting. [Wikipedia](#)
Representation learning. [Wikipedia](#)

Probe. [Wikipedia](#)

Activation patching. [Wikipedia](#)

Sparse autoencoder. [Wikipedia](#)

Sparse coding. [Wikipedia](#)

Mechanistic interpretability. [Wikipedia](#)

Multimodal learning. [Wikipedia](#)

Intelligent agent. [Wikipedia](#)

Software agent. [Wikipedia](#)

Data provenance. [Wikipedia](#)

Formal verification. [Wikipedia](#)

Model checking. [Wikipedia](#)

Knowledge graph. [Wikipedia](#)

Graph neural network. [Wikipedia](#)

Reinforcement learning. [Wikipedia](#)

Recommender system. [Wikipedia](#)

Time series. [Wikipedia](#)

Model card. [Wikipedia](#)

Out-of-distribution detection. [Wikipedia](#)

Closing perspective

Explainability remains an unfinished discipline, and that incompleteness is productive. The field is learning to distinguish an attractive explanation from a tested one, model-level dependence from world-level causation, traceability from mechanism, and broad trust from bounded evidence. The durable direction is not toward one perfect explainer. It is toward AI systems whose important claims can be inspected, challenged, replayed, and revised as the models and the world around them change.